

Software-as-a-Graph: When Does Graph Learning Improve Cascade-Impact Ranking in Publish–Subscribe Systems?

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Publish–subscribe middleware obscures how failures propagate, yet architects must identify critical components before deployment, without telemetry. We present Software-as-a-Graph (SaG), which derives explicit dependency graphs from deployment manifests to investigate when graph learning improves cascade-impact ranking. We compare graph neural networks, hybrids, and analytical and learned simulator approximations on twelve synthetic architectures under leave-one-scenario-out evaluation against three failure simulators, and zero-shot on five hand-authored models of open-source systems. A reference criterion distinguishes rankings that restate a simulator’s propagation rule from those that predict it.

On a reachability simulator, graph attention on the derived dependency graph reaches Spearman’s $\rho = 0.748$, +0.072 above the same model with reverse edges on the raw multigraph, whereas on the raw multigraph the registered primary contrast against a training-free baseline was null. It is not distinguishable from counting direct dependents ($\rho = 0.764$), and its gain survives removing input features that compute part of the simulator. For a queue-flow simulator requiring 12.7 CPU-hours to label the corpus, a rate-weighted first-order approximation reaches $\rho = 0.830$ in milliseconds, matching or exceeding a learned approximation trained on simulator labels ($\rho = 0.799$; exploratory). Learners that start from that approximation do not improve on it. Learned models transfer zero-shot better than the training-free baseline ($\rho \approx 0.81$ vs. 0.53), although dependency counts rank higher.

All three simulators admit such approximations, so no regime where learning could exceed one was exercised; findings are conditional on simulation fidelity. Representation mattered more than model complexity, so learned methods should be evaluated against analytical alternatives aligned with the simulator.

Keywords: Dependency graphs, cascading failures, publish–subscribe, graph neural networks, graph learning, software architecture, dependability

1. Introduction

1.1. Motivation and Problem

Large distributed systems now often use asynchronous publish-subscribe (pub-sub) middleware. Examples include ROS 2 for autonomous driving [1], Apache Kafka for enterprise event streams [2], DDS in cyber-physical systems [3], MQTT in IoT [4], and cloud-native microservices [5, 6]. Pub-sub separates producers and consumers by space, time, and synchronization [7]. Instead of direct references, components communicate via topics and brokers, and Quality-of-Service (QoS) policies set at deployment control reliability, durability, priority, and deadlines.

This decoupling also hides how failures spread. Outages, blocking and backpressure travel through brokers, shared topics, colocated hosts and shared libraries [8, 9]: in sequential cascades a slow subscriber fills a broker queue and starves its publishers [10], and in simultaneous blasts a library crash or host outage

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

takes down every colocated service at once. Neither is visible in architecture diagrams or call graphs. The cheapest time to address these risks is before deployment, during design and continuous integration [11, 12], when no telemetry exists, so critical components must be identified from configuration manifests alone.

Current practice leaves a gap between service-level static analysis and system dependability: a system can have no defect in any service yet fail through hidden single points of failure or mismatched QoS contracts [13, 14]. ATAM depends on manual input [15], static code analysis inspects services in isolation [16, 17], and chaos engineering [18], fault injection [19] and dependency tracing [20] need a running system and, in CI, considerable computation and energy [21, 22, 23]. Graph representations have long supported reliability and defect analysis [24, 25], and graph learning has recently been applied to network criticality [26, 27] and microservices [28]. It remains unclear whether such models’ performance comes from the learning algorithm or from the dependency representation it reads, and when learning adds value beyond analytical methods derived from the same dependencies. Software-as-a-Graph (SaG) addresses this question.

1.2. The Software-as-a-Graph (SaG) Approach

SaG models an event-driven architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts and Shared Libraries (Figure 1; Section 3.1), derives an explicit DEPENDS_ON dependency graph through publish–subscribe rules (Section 3.3), and supports analytical, hybrid and learned cascade-impact rankings on it (Sections 4 and 5.2).

Rankers never read simulator output. Labels come from three simulators on the raw topology (Section 4.3) that encode different failure notions: structural reachability (I^*), discrete-event queue saturation (I_{dyn}) and multi-criteria fragmentation (I_{comp}); every ranker is evaluated against all three. Because simulators and rankers read the same architecture, some analytical rankings restate a simulator’s rule; a reference criterion (Section 4.4) reports these as measures of circularity, not as predictors. All conclusions are conditional on simulation fidelity (Section 7.5).

The simulators give learning different roles. The reachability simulator ranks a corpus architecture in under a second, so a learned approximation of it has no practical use; we study it to calibrate the method, measuring how much of a simulator a learner and a restatement of its rule each recover. The queue-flow simulator is expensive, so approximating it is a genuine surrogate-modelling problem, and that is where we test learners that start from the analytical approximation (Section 6.1).

1.3. Findings in Brief

Explicit dependency representations provide most of the predictive signal for the reachability and queue-flow simulators.

Making dependencies explicit enables graph learning. On the raw multigraph every relation points away from Applications, so message passing never reaches the scored nodes, and the registered primary contrast (a heterogeneous graph transformer against the training-free baseline) was null ($\Delta\rho = +0.069$, $p = 0.266$). On the dependency graph, graph attention reaches Spearman $\rho = 0.748$ against the reachability simulator (GAT-P-QoS), against 0.635 on the raw multigraph. Edge direction does not account for the gain (the derived graph adds +0.072 over a raw-graph GAT that passes every edge both ways, Holm $p = 0.014$), nor do the features that compute part of the simulator: with all of them removed, the dependency-graph GAT beats an equally stripped reverse-edge control by +0.231 (Holm $p = 0.0068$), while the raw-graph learners collapse.

The same representation enables simple analytical rankings that the learned models do not exceed. Counting direct dependents, publish–subscribe afferent coupling, reaches 0.764 and is not distinguishable from the best learned model, which approaches it through its degree features. On the queue-flow simulator (12.7 CPU-hours to label the corpus), a rate-weighted first-order approximation reaches 0.830 in milliseconds and exceeds a gradient-boosted approximation trained on the simulator’s labels (0.799; exploratory); declared rates and payload sizes, not QoS policies, carry the extra signal.

Learning adds measurable value only relative to a weak comparator. Hybrids that correct a closed-form prior outperform the training-free baseline but not their own base learners, so the gain belongs to the baseline’s weakness. Learners that start from the rate-weighted approximation do not improve on it: as an extra input it leaves a gradient-boosted model at its value, and as a prior it leaves a graph attention network

below it (-0.018). Learned models transfer zero-shot to five stylized models of open-source systems better than the training-free baseline ($\rho \approx 0.81$ vs. 0.53), mainly by separating inert from active components, while dependency counts rank higher still and ordering among active components remains weak ($\rho_{>0} \leq 0.342$). All three simulators are well approximated by low-order functions of the dependency graph, so the benchmark did not exercise a regime in which learning could exceed an aligned analytical ranking.

In practice, the results favor training-free dependency analysis for routine use. Recovering 80% of the true critical set requires reviewing about 40–45% of Applications, so rankings suit prioritized review rather than blocking gates (Section 7.4).

1.4. Research Questions

Establishing when learning is unnecessary is as informative as showing when it succeeds. Software engineering research has repeatedly shown that simple metrics can match the performance of more complex models when the right information is available [25, 29, 30, 31]. Analytical rankings can also be easier to explain, validate, and run in Continuous Integration and Continuous Deployment (CI/CD) pipelines. We therefore evaluate learned models against analytical rankings derived from the same dependencies, not only against structural baselines. Four research questions guide the study:

- **RQ1 (Ranking accuracy):** *How accurately do analytical, hybrid, and learned approaches rank components by cascading-failure impact on unseen architectures, and how close do they come to reference rankings that restate a simulator’s propagation rule?*
- **RQ2 (Sources of predictive performance):** *At matched model capacity, which factors contribute most to ranking performance: the dependency representation, degree information, relation typing, Quality-of-Service (QoS) information, or the model family (attention or sum aggregation; graph neural network or gradient-boosted trees)?*
- **RQ3 (Transfer):** *How well do learned models trained on synthetic architectures transfer zero-shot to stylized models of five open-source systems, hand-authored by a single modeler, and does their ordering of components that actually propagate failures hold?*
- **RQ4 (Cost):** *What do analytical and learned approaches cost at CI/CD time in latency and estimated energy, which stage dominates, and how does this compare with running each simulator directly?*

The evaluation follows an analysis plan pre-specified in the replication repository, where each entry carries a commit timestamp; it was not lodged with a third-party registry, and “registered” in this paper means pre-specified in that plan. Only the plan’s two co-primary contrasts are confirmatory, and both were null. We registered the matched 2×2 , the hybrids, the reference counts, the dependency-graph learners (whose contrasts were declared exploratory), the full-population queue-flow labels, the sensitivity arms and the controls of Section 6.2 as planned extensions. We registered each before its own tests ran but after we knew the main result, and we report them as secondary results; no confirmation corpus generated after the analysis was frozen has yet tested them (Section 7.5). The rest of the analysis is exploratory (Section 5.3). One change from the plan, not applying the registered nested hyperparameter selection, is disclosed and tested. The replication repository records the revision history and shows how these four research questions fit into the plan.

1.5. Contributions

This paper makes five contributions:

1. **Dependency derivation for publish–subscribe architectures** (Section 3). A method for deriving explicit dependency graphs from deployment manifests using typed architectural rules, extending the Application-level rule of [32] with library-mediated dependencies (Rule 5); infrastructure rules are defined but not evaluated.
2. **An evaluation guideline for simulator-labeled benchmarks** (Section 4.4). An operational criterion, a truncation of the simulator’s own computation, that identifies rankings restating a simulator’s propagation rule, so that learned rankers are compared with them rather than credited for agreement they encode.

3. **A controlled comparison of analytical, hybrid, and learned ranking methods** (Sections 5.2 and 6.1–6.3). A study that separates the effects of dependency representation, degree information, relation typing, QoS information, and model family on cascade-impact ranking.
4. **Analytical and learned approximations of an expensive simulator** (Sections 5.2 and 6.1). A rate-weighted first-order approximation of a queue-flow simulator (Eq. (7)), compared with learned approximations trained on the simulator’s labels.
5. **A reproducible benchmark, cost analysis and practitioner guidance** (Sections 5.1, 6.4 and 7.4). A reproducible benchmark of seventeen architectures with three simulators, a like-for-like cost analysis, and recommendations for pre-deployment cascade-impact analysis.

A previous conference paper [32] introduced the publish–subscribe multigraph, the Application-level subscriber-to-publisher dependency (Rule 1 of Table 2), and a closed-form betweenness–articulation score similar to the training-free baseline (Eq. (5), unweighted and with weights 0.7/0.3), which it validated against a reachability-loss simulation on synthetic topologies and two ROS 2 benchmarks. This paper adds the Library and infrastructure rules, QoS weighting and the QoS edge encoding, the reference criterion, the learned and hybrid rankers, the analytical and learned simulator approximations, LOSO and zero-shot evaluation, the second and third oracles, the matched control, and the cost analysis. The closed-form score family validated in [32] is the weakest analytical ranker in the present evaluation, which adds reference rankings, the Application population and three oracles; the present results supersede that paper’s implicit recommendation of it.

Section 2 reviews related work. Sections 3 and 4 present the model and the rankers. Sections 5 and 6 cover the evaluation. Section 7 discusses the findings and threats, and Section 8 concludes.

2. Related Work

2.1. Dependability and Failure-Impact Analysis of Distributed Systems

Runtime dependability methods like broker clustering, backpressure, autoscaling, failover, and chaos engineering [18] require a running cluster, can interrupt service, and consume significant computing resources. This resource use is a major concern in green software engineering and Green AI [21, 22, 23], where the carbon footprint of large execution clusters in continuous integration encourages lightweight, energy-efficient static alternatives. Pre-production methods, such as service-level fault injection testing [19] and microservice dependency localization with PageRank (MicroRank [20]), offer lighter options before deployment. Benchmark platforms with cataloged, replayable faults, like Train-Ticket [33], help measure the impact of failures on a running deployment, which this study has not yet validated (Section 7.6). Architecture-based reliability prediction has a long history. Yacoub and Ammar’s method for architecture-level reliability risk analysis [24] was one of the first to combine component dependency graphs with complexity and failure severity to rank components by risk. State, path, and additive reliability models reviewed by Goseva-Popstojanova and Trivedi [34] are classic examples, as is the Palladio Component Model [35].

Error propagation has also been analyzed at the architecture level. Abdelmoez et al. [36] estimated error propagation probabilities between architectural components, and Popic et al. [37] added them to architecture-based reliability models of component-based systems. Cortellessa and Grassi [38] analyzed how error propagation affects system reliability, and Hiller et al. [39] profiled how data errors propagate across software modules (EPIC). In safety-critical systems, model-based failure analysis is standard practice, including Hierarchically Performed Hazard Origin and Propagation Studies (HiP-HOPS [40]) and the Architecture Analysis & Design Language (AADL) Error Model Annex [41]. These frameworks derive propagation paths from structural descriptions, serving as conceptual precursors to SaG’s dependency projection. However, these approaches need component failure-mode annotations, propagation probabilities or operational profiles, which deployment manifests do not carry; SaG asks a narrower question using only the manifests.

Telemetry-based methods find faults in running microservices. Examples include Seer [42], MicroRCA [43], and recent GNN-based root-cause frameworks like Eadro [44], which Zhang et al. [45] thoroughly review. In synchronous microservices, failures can spread because of thread-pool exhaustion, RPC timeouts, and retry storms [33]. In publish-subscribe systems, issues spread through queue saturation and message

starvation. All these methods need data from a live system, but SaG works before any code is run. Production traces also show that real dependency graphs are large and dynamic [46], and that some outages, such as metastable failures, are sustained by load feedback rather than by the dependency structure that failed [47, 48]; a structural ranking cannot see such failures. SaG’s focus is similar to change impact analysis [49] and dependency management in microservice fleets [50]. Ranking by impact is related to learning-to-rank defect prediction [51], which explicitly optimizes the ranking measure, as our listwise loss does (Section 4.2).

Rankings of failure impact are often evaluated against a failure model or simulator rather than against observed outages. Such evaluation creates an interpretive challenge: a ranking may agree with a simulator because it predicts the simulator’s behavior, or because it encodes the simulator’s propagation rule. The underlying concern is not new. Simulation methodology treats a model used as ground truth as something to be validated in its own right [52]; in network science, simple neighborhood counts are known to track the outcome of spreading processes closely when propagation is short-range [53]; and in machine learning, models are known to exploit shortcuts that encode the labeling rule rather than the intended task [54]. What we add is an operational check for simulator-labeled ranking benchmarks: the ranking obtained by truncating the simulator’s own computation, against which learned rankers are compared (Section 4.4).

2.2. Static Analysis and Architectural Dependency Metrics

Static code analysis (SCA) tools like SonarQube [16] measure complexity [55], cohesion, and coupling [17] within individual services to identify modules likely to have defects [56, 57]. Software engineering defect prediction has long compared complex graph measures with simple structural metrics. For example, Zimmermann and Nagappan [25] studied network analysis on dependency graphs, while Premraj and Herzig’s replication study [29] showed that simple code and coupling metrics frequently perform as well as network measures. SCA cannot detect inter-service messaging, broker saturation, or issues that spread across hosts. Architecture recovery tools rebuild system-level structure from code [58], and HAROS [59] and ROSDiscover [60] recover and check the run-time architecture of ROS systems before launch, which is the kind of extraction SaG’s models would need on real systems. SaG’s static system analysis (SSA) uses the declared topology instead, spreading code-level metrics across architectural dependencies. Architecture-level coupling metrics count a service’s consumers, such as afferent coupling [61], the Absolute Importance of the Service [62], and service fan-in [63]. In publish–subscribe systems, these counts are not simply a single edge lookup, since consumers reach producers only via topics, brokers, and shared libraries. They require a typed two-hop query (publisher to topic to subscriber), which SaG’s dependency rules define clearly. Because this count corresponds to the first propagation wave of the reachability oracle used here, this paper reports it as a reference that restates the oracle, not as a predictor (Section 4.4). This helps teams find structural anti-patterns [64, 65] and architectural technical debt [66] in CI/CD [67]. These studies suggest that much of the information about architectural criticality can be recovered directly from explicit dependency structure. They leave open whether learned methods add predictive information beyond such dependency-based analyses.

2.3. Multi-Criteria Weighting

Combining QoS policies or failure effects into a single, auditable weight is a multi-criteria decision problem. The Analytic Hierarchy Process (AHP) [68] is the standard method, and SaG uses it for the topic weights in Section 3.2. AHP’s consistency ratio can find inconsistent judgments, but it does not detect matrices that have been filled in to match a chosen answer, so the weights of the composite oracle (Section 4.3) are reported as declared rather than elicited.

2.4. Graph Learning on Dependency Structures

Closed-form indices [69, 70, 71, 72], including one used as this study’s training-free baseline, and cascade models of network robustness [10, 8, 9] assume homogeneous, usually undirected graphs. This means a single untyped score can mix up different elements like topics, libraries, and hosts. Learned node-importance methods, such as FINDER [26], DrBC [27], and PowerGraph [73], also assume homogeneity, as does learned approximation of betweenness and closeness [74]; GENI [75] estimates node importance on heterogeneous knowledge graphs, the closest learned analogue to a typed ranker. Homogeneous GNNs like Graph Attention

Networks (GAT [76]) ignore relation identity unless it is added as a feature. Softmax attention averages over neighbors and cannot count them, while sum aggregation can [77, 78]; theoretically, standard message-passing GNNs are strictly limited in counting substructures without explicit features [79]. Since the main reference here is neighbor count, this difference matters (Section 6.2). Heterogeneous GNNs, like the Heterogeneous Graph Transformer (HGT [80]), learn transformations specific to each relation. However, extensive benchmarking by Lv et al. [81] demonstrated that heterogeneous GNNs often fail to outperform well-tuned homogeneous baselines like GAT, directly paralleling our typing ablation findings. More broadly, Shchur et al. [82] showed that GNN rankings are sensitive to data splits and tuning, while Huang et al. [83] showed that label propagation combined with shallow models can outperform GNNs on transductive node classification. We test both types with the same capacity (Section 6.2). Khodabandeh et al. [28] use graph attention on microservice call graphs to predict upcoming interactions. In contrast, we predict the effect of removing a node from a declared topology. These studies show that graph learning can exploit structural information in software and network graphs. Their evaluations typically compare learned models with selected baselines; fewer examine whether performance originates from the graph representation or from the learning algorithm applied to it.

2.5. Positioning of This Study

This study builds on findings that simple baselines can perform as well as more complex learning methods [30, 31], and that network measures on dependency graphs [25] add little over simple coupling metrics once those are included [29]. In line with Green Software Engineering and Green AI principles [21, 22, 23], the cost of learned models in continuous integration should also be weighed against that of simple static analyses.

The study differs from prior work in three ways. First, it derives explicit dependency graphs from publish–subscribe deployment manifests rather than starting from an existing architectural graph. Second, it evaluates analytical, hybrid, and learned ranking methods using the same representation and asks where their ranking performance originates. Third, it applies an operational reference criterion, a truncation of the simulator’s own computation, to separate rankings that restate a simulator’s rule from those that predict its outcomes. Rather than asking whether graph learning can rank cascade impact, the study asks when it adds measurable value beyond rankings derived directly from the explicit dependency structure. More broadly, the study contributes to understanding the limitations of AI techniques for reliability analysis by identifying when apparent predictive accuracy reflects restatement of the evaluation simulator rather than learned skill.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 shows the full SaG framework architecture. The shared front end handles Architecture-as-Code manifests, builds a typed multigraph, derives a logical dependency layer that makes runtime failure paths explicit, and extracts typed node properties. This representation supports graph-learning pipelines and makes software dependencies explicit, so analytical, hybrid, and learned cascade-impact rankings can be computed from the same architectural representation (Sections 4 and 5.2). This common representation enables controlled comparisons between methods that differ in complexity but operate on identical dependency information.

3.1. Multigraph Definition

A distributed system is described as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$ holds the five entity types $\mathcal{T}_V$ of Table 1; V_{host} denotes physical or virtual *Execution Hosts*.
- E is the set of directed edges, and $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ assign entity and relation types.

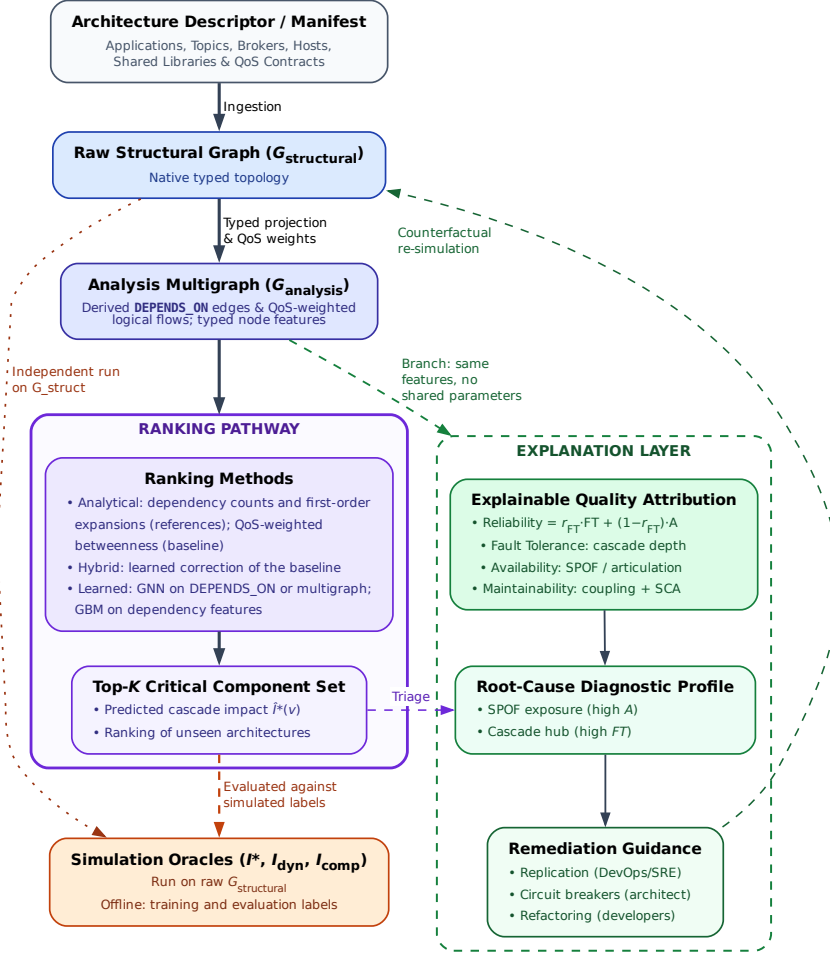


Figure 1: End-to-end architecture of the SaG framework. The ranking pathway runs down the center: manifest ingestion, typed structural multigraph ($G_{\text{structural}}$), DEPENDS_ON logical dependency projection (G_{analysis}) with typed node properties, the ranking methods (analytical, hybrid, and learned; Figure 3), and the ranked critical set. The same dependency graph supports all three ranking methods. The dashed edge marks the simulation oracles, which operate strictly on $G_{\text{structural}}$, provide training and evaluation labels offline, and do not participate in inference. The explanation layer is a proposal outside this study (Supplementary Section S26); no ranker output flows into it.

- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ weight entity criticality along with connection strength. In the implementation evaluated here, w_V of an Application is a power mean of the QoS weights $w(t)$ (Section 3.2) of the topics it publishes or subscribes to, and a Library takes the largest weight among its own topics and its consuming Applications, amplified by its fan-out. A code-quality penalty (CQP), computed from lines of code, cyclomatic complexity, instability and cohesion, does not enter w_V ; it and its four inputs appear only as node features of the learned rankers (Section 3.5).

The raw multigraph is denoted $G_{\text{structural}}$. The simulation oracles run on it, and the raw-graph learners pass messages over its edges. The analysis graph G_{analysis} is $G_{\text{structural}}$ augmented with the derived DEPENDS_ON edges (Section 3.3) and the typed node properties computed from them (Section 3.5). Its Application–Library DEPENDS_ON projection is the graph the analytical rankings and the -P learners read. Supplementary Section S1 lists the notation.

Scope of Evaluation and Entity Roles. While the multigraph formalizes five entity types, this study concentrates its empirical study specifically on *Applications* (V_{app}). An *Application* (the term used by OMG DDS and ROS 2) corresponds to a microservice container, service agent or communicating process: the developer-authored code that changes in CI/CD, and hence the target of pre-deployment triage. Brokers,

Topics, Execution Hosts and Libraries are modeled because failures propagate through them, but ranking them is a provisioning question rather than a commit-time one, so the evaluation ranks Applications only; brokers and hosts remain an extension (Section 7.6).

Table 1: Entity types and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	/sensor/lidar, orders.payment.completed
Execution Host (V_{host})	Physical or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO / SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component publishes to / consumes from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on host
CONNECTS_TO	Host $\rightarrow$ Host	Network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library

3.2. Quality-of-Service Link Weighting

A link’s strength is conceptually determined by its Quality-of-Service (QoS) contract. For example, a RELIABLE topic with TRANSIENT_LOCAL durability creates a stronger connection between services than a BEST_EFFORT telemetry stream. Each topic t has an overall weight $w(t) \in (0, 1]$ that combines its QoS policies (reliability, durability, priority) with payload size and publication frequency. We set the reliability, durability, and priority weights employing an Analytic Hierarchy Process (AHP) pairwise-comparison matrix, which we defined independently and did not adjust to fit a target vector ($CR = 0.016$, non-degenerate; Supplementary Section S6).

The evaluation tests whether this contractual information improves cascade ranking, and it does not: declared QoS policies carried no measurable signal on either the reachability or the queue-flow simulator (Sections 6.1 and 6.2). For the reachability simulator this null is largely fixed by design, because I^* barely reads QoS (Section 4.3); the queue-flow simulator does read QoS policies, so there the null may be informative, although its sensitivity to them was not measured. The weighting is retained for completeness; the dependency counts reported in this paper are unweighted and do not use $w(t)$.

3.3. Logical Dependency Projection (DEPENDS_ON)

Structural edges do not directly show how failures spread. For example, a subscriber depends on a publisher, but pub-sub topologies have no direct edge between them. SaG therefore derives an explicit semantic relation, DEPENDS_ON, from the dependent to the dependency: if the target fails, the source is affected. The derived edges expose the architectural pathways through which failures may propagate, and the resulting dependency graph is the common input for analytical rankings, hybrid methods, and graph-learning models. Table 2 lists the rules.

Table 2: The DEPENDS_ON logical dependency projection rules. [†]Defined for completeness; not exercised by the Application-level evaluation.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic)	$1 - \prod_{t \in T} (1 - w(t))$
2 [†]	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3 [†]	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_E(u \rightarrow v)$
4 [†]	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_E(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6 [†]	broker_to_broker	Broker $\leftrightarrow$ Broker (shared fault-domain colocation, symmetric)	$w_V(\text{host})$

Rules 1 and 2 combine topics T that connect a pair using probabilistic union [84], so that having multiple failure paths increases the connection strength. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [85]. **Sequential cascades and simultaneous blasts.** Rule 1 describes sequential cascades, where if a publisher fails, its subscribers are affected because they do not receive messages. Rule 5 describes simultaneous blasts,

where if a library crashes, all applications that depend on it fail at the same time. Neither mechanism is stated explicitly in a deployment manifest; each rule makes one of them visible. Shared-library dependencies are included because they create simultaneous blast failures that message-flow topology alone does not represent. Libraries that publish or subscribe are also endpoints for Rule 1, and an Application can reach a library's topics through Rule 5. **InDeg**, **Reach**, **Topo-QoS** and the graphs the **-P** learners read apply Rule 1 to direct subscriptions only (Eq. (2)). The node-property extraction, which produces the in-degree feature every learner receives, additionally follows up to three **USES** links, so an Application that subscribes through a library it uses counts as a Rule 1 dependent in that feature only. The two counts differ for 940 of the 1,321 Applications, the feature always being the larger, and agree in rank at $\rho = 0.55$ –1.00 per fold. Rules 2, 3, 4, and 6 (host failure domains and broker routing) are defined for completeness and are not used in the Application-level evaluation.

Remark 1 (the dependency count is a typed two-hop count). Let G_{flow} be the Application–Library projection (Rules 1 and 5) and v an Application. By construction,

$$\text{InDeg}(v) = |\{u \neq v : \exists t \in V_{\text{topic}}, (v, t) \in \text{PUBLISHES_TO} \wedge (u, t) \in \text{SUBSCRIBES_TO}\}|. \quad (2)$$

Since Rule 5 edges end at Libraries, every edge into an Application is a Rule 1 edge $u \rightarrow v$, which exists exactly when $u \neq v$ (an Application or a Library) directly subscribes to a topic that v publishes; in the presence of multiple parallel topics between the same pair, parallel topics collapse into a single directed dependency edge in G_{flow} .

The right side of Eq. (2) represents publish–subscribe afferent coupling (fan-in, or the Absolute Importance of the Service [61, 62]), and is a typed two-hop query on the raw multigraph. **InDeg** can be computed directly without projection; the projection defines which typed query to use. We checked this equality on all seventeen corpus graphs, with a maximum absolute difference of 0. We measure the value of the derivation separately: including Rule 5 raises the agreement of transitive reach with I^* by +0.058 (9 of 12 folds, Holm $p = 0.0068$; Section 6.1). Since the primary oracle's first propagation wave matches this set (Section 4.4), **InDeg** and its transitive version **Reach** are used as reference points in this paper, not as predictors.

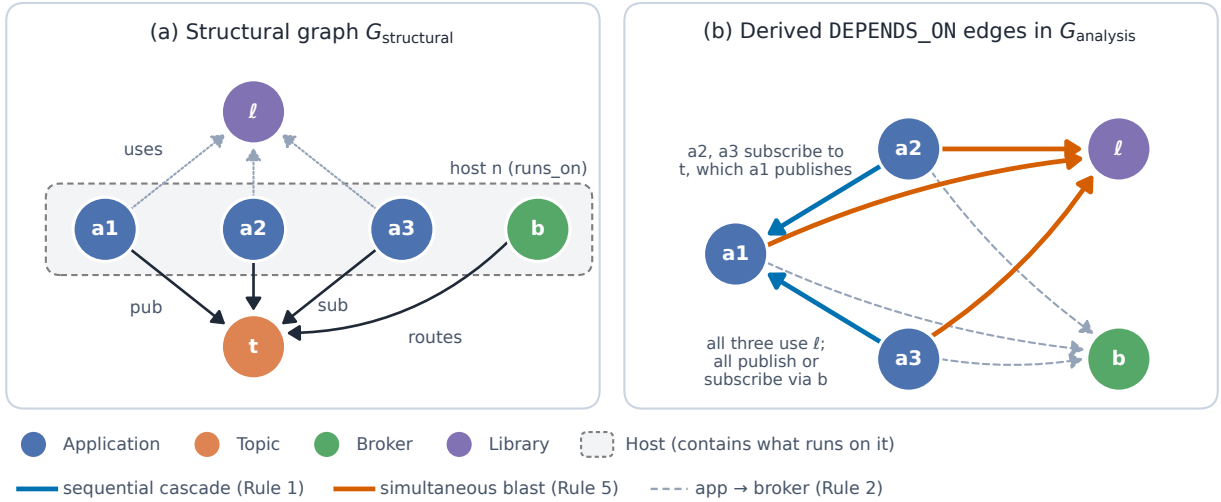


Figure 2: Running example. (a) Three applications share topic t (routed by broker b) and library ℓ , and all run on host n . No structural edge joins two applications. (b) The derived **DEPENDS_ON** edges make the hidden dependencies explicit: subscribers a_2, a_3 depend on publisher a_1 (Rule 1), all applications depend on ℓ (Rule 5), and each application depends on the broker (Rule 2). Simulation oracles run on view (a); the analytical rankings and the **-P** learners read view (b), and the raw-graph learners pass messages over view (a) with node features computed from (b).

3.4. Dual Graph Views

The **structural graph** $G_{\text{structural}}$ is the raw deployment topology. The **analysis graph** G_{analysis} adds the **DEPENDS_ON** edges and code metrics (see Figure 2). Every ranker's node features are computed on

G_{analysis} ; the learners pass messages either over the edges of $G_{\text{structural}}$ or over the **DEPENDS_ON** projection (Table 4), and no ranker reads simulator output (Section 4.4). The two views also differ for a learner: on $G_{\text{structural}}$, every relation points away from Applications, so forward message passing does not reach them. In the **DEPENDS_ON** graph, each Application receives messages from its dependents (Section 6.2). The raw multigraph is therefore also a comparison point in the evaluation: it exposes communication structure but not cascading dependencies. Because the two views also differ in edge direction, Section 6.2 adds a raw-graph learner that passes every message in both directions, so that dependency semantics can be separated from direction.

3.5. Typed Node Feature Encoding

The rankers use typed node properties from G_{analysis} . All five entity types share a base set of 18 normalized metrics (in $[0, 1]$): PageRank, Reverse PageRank, betweenness, closeness, eigenvector centrality, in-degree, out-degree, clustering, an articulation score (a continuous directed cut-vertex score that the implementation stores in two columns, the 18 including both), bridge ratio, node QoS weight, incoming and outgoing dependency weights, multi-path coupling, path complexity, fan-out criticality, and the Connectivity Degradation Index (CDI). We calculate four centrality features (PageRank, Reverse PageRank, betweenness, and closeness) over weighted edges, so they include QoS information. This means the so-called QoS-off condition is not entirely free of QoS effects. Several of these features compute part of what the reachability simulator computes. The in-degree feature and the incoming dependency weight w_{in} (a QoS-weighted version) are close to **InDeg**; CDI is the connectivity lost when the node is removed, a removal-and-reachability computation on G_{analysis} ; and multi-path coupling, reverse PageRank and the articulation score are also aligned with that simulator. Every published learner reads them, so the learners inherit part of the oracle’s circularity; Section 6.2 reports learners with all of them removed. Type-specific blocks add more features, extending the vector to 19–25 dimensions (Applications and Libraries add the code-quality penalty and its four static-code inputs; the full schema is in Supplementary Section S13). CDI measures structural connectivity loss using a fixed-scale breadth-first sample to keep computation time reasonable (Section 6.4). We discuss its sensitivity to hash-seeded tie-breaking in Section 7.5.

In this study, SaG therefore serves two roles. It provides a dependency representation from which analytical rankings can be computed directly, and it provides a graph on which learned and hybrid methods operate. The evaluation in Sections 5–7 examines how much ranking performance arises from this representation and how much from the models applied to it.

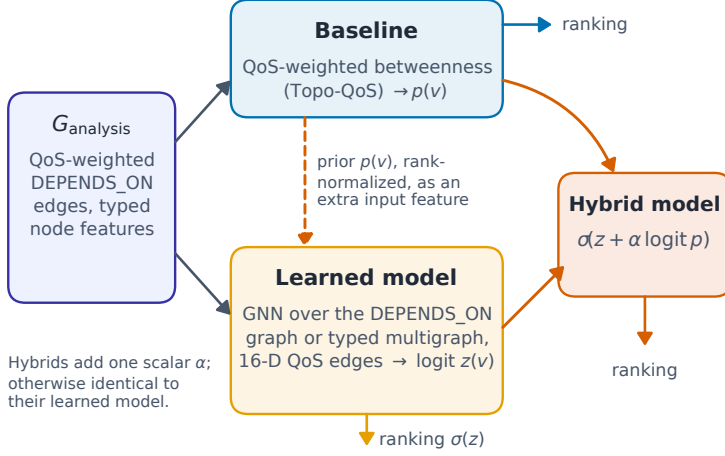
4. Ranking Methods, Simulation Oracles, and the Reference Criterion

The study compares analytical rankings derived directly from dependency structure (the training-free baseline **Topo-QoS**, QoS-weighted betweenness on the dependency graph, and the references of Section 5.2), hybrid methods that correct an analytical prior with a learned model, and learned methods: graph neural networks on the typed multigraph or the dependency graph, and gradient-boosted models on per-Application features (Figure 3). All are trained on or evaluated against simulation oracles that run on a separate graph view (Section 4.3). Full hyperparameters and training commands are in the replication repository’s experiment pages (Section 5.1).

4.1. Heterogeneous Graph Transformer and Attention Networks

The main learned ranker, **HGT-QoS**, is a three-layer Heterogeneous Graph Transformer (HGT) [80] built with PyTorch Geometric [86]. It uses a hidden dimension of $D = 64$ and $H = 4$ heads; on the dependency graph, which has fewer relation types, **HGT-P-QoS** uses width 100 so that its parameter count (430,680) stays within 1% of **HGT-QoS** (434,620). Entity-specific linear projections map raw node features $x_v \in \mathbb{R}^{19-25}$ into the hidden space, with $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. For each meta-relation $\langle \tau(u), \phi(e), \tau(v) \rangle$, attention uses type-parameterized keys, queries, and values, scaled by a learned per-meta-relation prior $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$; full layer-wise formulations are in Supplementary Section S2. HGT passes messages over its input graph and its transpose: $G_{\text{structural}}$ for **HGT-QoS** and the **DEPENDS_ON** projection for **HGT-P-QoS**. The

(a) Ranking methods on one analysis graph



(b) Simulator labels and evaluation

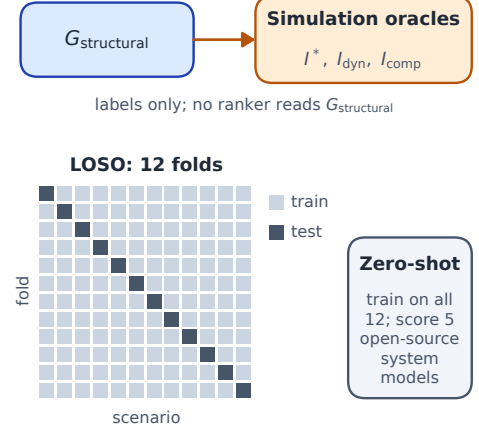


Figure 3: (a) SaG’s ranking methods read the analysis graph. The training-free baseline scores QoS-weighted betweenness; $p(v)$ is that **Topo-QoS** score, rank-normalized to $[0, 1]$ within the graph; the learned model outputs a logit $z(v)$. A hybrid model gives the learned model $p(v)$ as an extra input feature and adds a learned correction to it on the logit scale, $\sigma(z + \alpha \logit p)$, with one learnable scalar α . (b) Labels come from simulation oracles on the structural graph, which no ranker reads. We evaluate the methods by leave-one-scenario-out cross-validation over twelve synthetic architectures and zero-shot on five open-source system models.

GATs pass messages forward only, except the direction control **GAT-QoS-R**, which also passes every edge in reverse. In the raw multigraph $G_{\text{structural}}$, native relations point away from Applications (e.g., **PUBLISHES_TO**, **RUNS_ON**, **USES**), so forward message passing cannot propagate messages to the scored Application nodes, and forward GNNs on it collapse into per-node models over precomputed features (Section 7.2). The dependency graph directs edges from dependents to dependencies, so forward message passing aggregates information from each Application’s dependents. The homogeneous Graph Attention Network (**GAT-QoS**, and **GAT-P-QoS** on the dependency graph) uses a three-layer architecture with four attention heads and width $D = 288$ ($D = 296$ in the unweighted control for parameter matching), projecting all node types into a joint embedding space before applying homogeneous **GATConv** layers.

Each directed edge has a 16-dimensional vector. Index 0 represents the coupling weight, index 1 is the normalized path count, indices 2 to 8 are a one-hot encoding of the relation, and indices 9 to 15 are middleware QoS parameters such as reliability, durability, priority, transport departure mode, deadline constraints, and max-blocking time. The edge vector is projected and added before attention. Because indices 2 to 8 encode the relation, the “untyped” **GAT-QoS** receives relation identity as an edge feature. This matters only where messages reach Applications: on the raw multigraph forward GAT messages never do, and on the dependency projection every edge carries the same relation, so the encoding is constant.

4.2. Prediction Head and Training Objective

A composite head predicts the simulated cascade impact, supplemented by an auxiliary head $\hat{a}_1$ regressing on I_R^* , the reliability component that the same simulator run records beside I^* . The operational training objective combines mean-squared error regression with listwise and margin-ranking terms:

$$\mathcal{L} = \text{MSE}(\hat{I}^*, I^*) + 0.5 \cdot \text{MSE}(\hat{a}_1, I_R^*) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}}, \quad (3)$$

where $\mathcal{L}_{\text{rank}}$ is Listwise Maximum Likelihood Estimation (ListMLE) [87] over the permutation π induced by the simulated labels,

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\hat{s}_{\pi_i} - \log \sum_{j=i}^N \exp(\hat{s}_{\pi_j}) \right), \quad (4)$$

and $\mathcal{L}_{\text{pairwise}}$ is a margin-ranking loss with $\gamma = 0.05$ over pairs differing by more than γ . Tied labels, which affect about 31% of Applications with $I^* = 0$, enter π in input order, which is the generator’s creation order. This injects an arbitrary ordering among inactive nodes into ListMLE; the margin-ranking term ignores pairs separated by less than $\gamma = 0.05$, and a control that permutes node order before training tests whether the ordering leaks into the results (Section 6.2).

Models are trained with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4}) using cosine warm restarts for up to 300 epochs. We set early stopping at 60 epochs of patience on a 20% node-level validation split from the largest training scenario of each fold (when the corpus’s largest scenario is held out, the next largest takes its place), using five fixed seeds ($\{42, 123, 456, 789, 2024\}$); while this node-level split is transductive with respect to that scenario’s internal graph, all reported evaluations are strictly inductive under leave-one-scenario-out cross-validation. We set hyperparameters and loss coefficients before the first sweep and kept them the same across all experiments. This approach differs from the registered plan, which called for selecting every hyperparameter using nested inner-LOSO selection (two-fold inner holdout). Instead, the published results use one fixed configuration, chosen during the main experiments to keep computation manageable across seeds and folds. We explain this difference in Section 5.3. We ran the registered selection later as a sensitivity analysis on the eight-configuration stage-1 grid (varying depth and rank normalization of features and labels). That harness also holds one training scenario out for early stopping, so it trains on one scenario fewer and stops on a different signal; it therefore compares two protocols, not only two hyperparameter choices, which is why it does not reproduce the published scores on the folds where it selected the published configuration (Section 6.2). The main null finding does not change under nested tuning.

4.3. Simulation Oracles

Labels come from three failure simulators (“oracles”), which define different notions of cascade impact rather than observations of real failures and run on the raw multigraph $G_{\text{structural}}$:

- **Primary reachability cascade oracle (I^*):** Crashes component v , propagates outages through dependent topics, brokers and links via breadth-first traversal on $G_{\text{structural}}$, and computes the mean fractional feed loss across intact subscribers. Feed loss is scaled by a declared, uncalibrated QoS severity ladder ($\times 1.2$ RELIABLE, $\times 1.15$ high priority, $\times 1.05$ medium) and clamped to $[0, 1]$. Each propagation wave is processed in a seeded random order, which decides ties at the propagation threshold; the label is the mean over five seeds. Disabling QoS scaling leaves the Application ordering largely intact ($\rho = 0.965$ across the twelve folds), and substituting durability-aware rescaling moves it less still ($\rho = 0.977$), showing that I^* is predominantly a topological reachability metric.
- **Queue-flow discrete-event oracle (I_{dyn}):** A discrete-event SimPy [88] simulation of message emission rates, queue saturation and network latency that measures the drop in delivered message rate for surviving consumers, using declared rates, payload sizes and QoS contracts that I^* ignores. QoS policies act on the queues (drop order on overflow, durable replay, priority, deadlines and lifespans, history depth); no run with them disabled was made, so its sensitivity to QoS is not quantified. Its mechanism differs from reachability, but its first-order effect, consumers losing a failed publisher’s messages, is again subscriber loss, and it models configured parameters, not operating-system dynamics. It provides labels for all 1,321 Applications, averaging over five seeds, and takes 12.7 CPU-hours for the full corpus, compared to seconds for I^* . A sensitivity check restricted to the first 30 Applications per fold in lexicographic order is reported in Supplementary Section S40 (Table S63). I_{dyn} agrees with I^* at a mean $\rho = 0.711$ $[0.625, 0.794]$. Across seeds, a single I_{dyn} run agrees with another at $\rho = 0.43$ – 0.96 per fold; the five-seed mean used as the label has an estimated reliability of $r = 0.79$ – 0.99 (Spearman–Brown), so no ranker can agree with it by more than $\sqrt{r} \approx 0.89$ – 0.996 per fold. Every ranker in Table 6 stays below even r on every fold; per-fold values are in the replication package.
- **Composite multi-criteria failure oracle (I_{comp}):** An auxiliary benchmark uses the Validate-stage failure simulator. We run it on all 1,321 Applications with QoS weighting and a primed baseline flow. Each removal is scored as a linear combination of four degradation components: $I_{\text{comp}}(v) = 0.35 \text{RL}(v) + 0.25 \text{FR}(v) + 0.25 \text{TL}(v) + 0.15 \text{FD}(v)$, representing Reachability Loss (RL), Fragmentation (FR; 0.70 structural component loss + 0.30 severity), Throughput Loss (TL), and

Flow Disruption (FD). The weights are declared, not elicited, so I_{comp} serves as a declared multi-objective stress test of fragmentation and throughput disruption rather than just subscriber loss. Since fragmentation and flow disruption occur with almost every removal, $I_{\text{comp}}(v) > 0$ for nearly every Application. I_{comp} is never used as a training label.

4.4. Input-Label Separation and the Reference Criterion

No simulation output is used as an input: features are computed on G_{analysis} and labels come only from the oracles (a regression test enforces this), although the raw-graph learners pass messages over the structural edges the oracles traverse.

Procedural separation does not remove construct overlap: a ranking may agree with a simulator because it reproduces the simulator’s rule rather than because it infers impact from independent evidence.

The reference criterion. We defined the criterion post hoc, after simple dependency counts performed strongly, and state it here in operational form. A ranking R is an *order- k reference* for an oracle O if R is what O computes when its propagation is truncated after k waves, with stated simplifications, from the same inputs O reads. Whether a ranking qualifies is therefore decided by derivation from the simulator’s definition, not by its score. For I^* : the first-order expansion (Eq. (6)) is the expected feed loss after one wave; **InDeg** is the support of that first wave, its size without the per-topic fractions (Remark 1; classical publish-subscribe afferent coupling [61, 62]); and **Reach** is the unweighted support of the untruncated cascade. For I_{dyn} , the rate-weighted expansion (Eq. (7)) is the delivered-rate loss after one wave, ignoring queueing. A reference measures how much of O its own rule recovers, a property of the oracle rather than predictive skill, so comparisons with it are not claims of superiority. All other rankers, namely the training-free baseline and the learned and hybrid methods, are *predictors*. For I_{comp} , no ranking is a truncation; its fragmentation and flow terms respond to removing high-degree and high-betweenness nodes, so **Topo-QoS** and raw degree are *term-aligned* with it. They are not references under the definition above, but their performance is interpreted with the same caution. How much of each oracle its own rule recovers is read directly from the reference rows: the first-order expansion reaches $\rho = 0.808$ on I^* , and the rate-weighted expansion 0.830 on I_{dyn} (Tables 5 and 6). The learned methods are not references by this definition, but this does not make them less circular: they are fitted to the oracle they are scored against and read features that compute part of it (Section 3.5). We therefore apply the same accounting to them, reporting each learner’s agreement with I^* after the first-order expansion is partialled out (Section 6.1), and learners with the oracle-aligned features removed (Section 6.2). Two analyses limit the circularity without removing it: every ranker is scored on all three oracles, and its agreement with I_{dyn} is also reported after removing I^* or the first-order term (Section 6.1). Neither replaces validation against real failures (Section 7.5).

5. Experimental Setup

The design asks whether ranking performance originates from the dependency representation or from the learning algorithm operating on it: RQ1 compares learned approaches with analytical rankings derived from the same dependencies, and RQ2 isolates the contributions of representation, degree information, oracle-aligned features, relation typing, QoS information and model family.

5.1. Corpus and Replication Package

The corpus includes 2,812 components from seventeen architectures (see Table 3). Twelve synthetic topologies make up the LOSO folds, covering areas like autonomous vehicles, financial trading, healthcare, industrial SCADA, smart-city IoT, telecom RAN, logistics, gaming, microservices, enterprise integration, and air-traffic management. Because all twelve topologies and their code metrics come from the same family, LOSO tests transfer across several parameter settings of that generator, not across unrelated industrial systems. We can regenerate each topology exactly from its configuration, and continuous integration checks this using a SHA-256 manifest.

The five open-source systems are hand-crafted models. Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and two meshes based on Online Boutique and Train-Ticket were each created by the first

Table 3: Evaluation corpus. The twelve synthetic topologies are the LOSO folds; we exclude the five open-source system models from all training and use them only for zero-shot transfer (Section 6.3). Counts are read from the committed topology files and verified in CI; Supplementary Section S15 gives per-scenario composition.

Dataset	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (12)	2,461	1,321	615	65	202	258	10,918
Open-source system models (5)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

author as typed multigraphs using public architectural documentation. We did not mechanically extract them from source code or container manifests, and no second modeler re-derived any of them, so modeler bias is possible and not quantified (Section 7.5). Brokers, QoS profiles, code metrics and host specifications are partly assumed. Online Boutique and Train-Ticket are RPC systems: we decomposed each synchronous call into a request topic and a reply topic mediated by broker channels (Online Boutique becomes a 22-application mesh with four brokers; Train-Ticket’s service-discovery registry acts as the broker), which keeps the bidirectional dataflow but drops timeouts, thread-pool exhaustion and synchronous backpressure (per-system composition in Supplementary Section S15). Consequently, no topology in this study comes from a real deployment manifest: SaG’s derivation is exercised only on generated and hand-authored inputs, and RQ3 tests transfer to stylized single-modeler models, not to real production systems.

Replication package. All datasets, harnesses, checkpoints, and result artifacts are stored on Zenodo (see Data Availability). In addition, the public repository provides documentation for each experiment, including its protocol, hyperparameters, `make` target, artifacts, and extended results (<https://github.com/onuralpyigit/software-as-a-graph/tree/f3352f0c7cdf66ff77845b5b69624db539e82ed5/docs/research/jss/experiments>).

5.2. Rankers and References

Table 4 groups the rankers as analytical, hybrid and learned; other training-free baselines (Degree-raw, PR-raw, RevPR-raw, Topo) and smaller or projection variants are in Supplementary Sections S42 and S34. Analytical rankings that restate a simulator’s rule are listed as references, not predictors. For GNNs, “-QoS” means the 16-dimensional QoS edge vector (Section 4.1) plus three QoS node columns (w , w_{in} , w_{out}), and “Hybrid-X” is model X corrected by the baseline prior. GAT and GAT-QoS are untyped GATs at HGT’s parameter budget, giving a 2×2 of typing and QoS channel. Learned and hybrid models use the native multigraph and the “-P” models the Application–Library `DEPENDS_ON` graph, with identical node features and labels. QoS-off arms set edge weights to 1 and zero the QoS node columns, although four centralities still use QoS (Section 3.5).

Training-free baseline. Topo-QoS is computed on the Application–Library `DEPENDS_ON` graph (Rules 1 and 5):

$$\text{Topo-QoS}(v) = 0.6 \cdot \text{BT}_w(v) + 0.4 \cdot \text{AP}(v), \quad (5)$$

where BT_w is betweenness over edge distances $d(e) = 1/(w(e) + 10^{-6})$, so tightly coupled edges attract shortest paths, and AP flags articulation points. Because of an implementation defect (an index mismatch when the articulation scores are read), the AP term is zero for every node, so the score that was executed, and that every table reports, is QoS-weighted betweenness alone. Restoring the term lowers **Topo-QoS** from 0.553 to 0.533, so the defect does not favor the reference counts; Table 5 reports the corrected value beside it, and hybrids retrained on the corrected prior are reported in Section 6.1. The comparator was fixed when the analysis plan was written, before the dependency counts had been analyzed, and continues the closed-form score family of [32]; in hindsight, afferent coupling would have been the natural comparator, and it is reported beside the baseline as a reference. The unweighted Topo baseline is reported in Supplementary Section S42.

Three training-free rankings restate the reachability oracle’s propagation rule and are reported as references. The first is the post hoc first-order expansion of I^* (Analytic I^*):

$$\hat{I}_1^*(v) = \sum_{t \in \text{pub}(v)} \frac{|\text{sub}(t)|}{|\text{pub}(t)|}, \quad (6)$$

Table 4: Rankers and references reported in the main text, grouped as analytical, hybrid, and learned. **-QoS**: QoS-weighted distances (**Topo-QoS**) or the 16-D QoS edge vector plus three QoS node columns (GNNs); **-P**: trained on the derived dependency graph; **Hybrid-X**: model X corrected by the **Topo-QoS** prior; **→dyn**: trained on I_{dyn} labels. Unsuffixed GNNs (**GAT**, **HGT**) serve as raw-multigraph confound controls ($G_{\text{structural}}$). Params: trainable parameters (— for tree ensembles and training-free rankings). References restate a simulator’s rule and carry no contrast against **Topo-QoS** (Section 4.4); ablation and control arms (**GIN-P-QoS**, **GBM-Feat**, the degree-free and oracle-aligned-feature-free learners, the node-order permutation, the reverse-edge control **GAT-QoS-R** and the corrected- and **InDeg**-prior hybrids) are reported in Section 6.2. [¶]**Topo-QoS** carries an implementation defect where the articulation point term is zero due to an index mismatch (so it acts as pure QoS-weighted betweenness, scoring 0.553; restoring the term lowers the score to 0.533, confirming that the defect does not disadvantage the baseline).

Ranker	Graph read	Typing / edge input	Params
<i>Analytical: training-free baseline (registered comparator)</i>			
Topo-QoS [¶]	App-Lib DEPENDS_ON	scalar $w(e)$; betweenness only (AP term zero; corr. 0.533)	—
<i>Analytical: references (no contrast; Section 4.4)</i>			
Analytic I^* (Eq. (6))	topic publish/subscribe sets	first-order subscriber loss	—
Rate-weighted (Eq. (7))	topic publish/subscribe sets	first-order loss weighted by declared rates	—
InDeg / Reach	App-Lib DEPENDS_ON	direct / transitive dependents	—
<i>Hybrid: learned correction of the Topo-QoS prior</i>			
Hybrid-GAT / Hybrid-HGT	$G_{\text{structural}}$ + features	as base + Topo-QoS prior	431,433 / 434,941
<i>Learned: dependency graph</i>			
GAT-P-QoS , HGT-P-QoS	App-Lib DEPENDS_ON	homogeneous / heterogeneous	429,992 / 430,680
<i>Learned: raw multigraph, typing × QoS channel at matched budget</i>			
GAT / GAT-QoS	$G_{\text{structural}}$ + features	homogeneous; none / 16-D	437,496 / 429,992
HGT / HGT-QoS	$G_{\text{structural}}$ + features	heterogeneous; 1-hot / 16-D	434,620
<i>Learned approximations of I_{dyn} (trained on I_{dyn} labels)</i>			
GBM-P-QoS →dyn	per-Application features (App-Lib DEPENDS_ON)	18 columns: 9 dependency counts, 9 QoS/rate	—
GAT-P-QoS →dyn	App-Lib DEPENDS_ON	homogeneous; 16-D	429,992

where $|\text{pub}(t)| \geq 1$ holds for all published topics $t \in \text{pub}(v)$ by construction (v publishes to t), preventing any division by zero. The second is **InDeg**(v), the number of direct dependents of v on the Application–Library **DEPENDS_ON** graph, which for an Application equals publish–subscribe afferent coupling and is exactly the set I^* ’s first wave reaches (Remark 1). The third is **Reach**(v), the number of its transitive dependents normalized by $|V| - 1$, which is the set a reachability cascade can visit. For the queue-flow simulator I_{dyn} , the corresponding reference is the rate-weighted first-order expansion

$$\hat{I}_{\text{dyn},1}^{\text{rate}}(v) = \sum_{t \in \text{pub}(v)} \frac{r_t}{|\text{pub}(t)|} |\text{sub}(t)|, \quad (7)$$

where r_t is the declared publication frequency of topic t . Eq. (7) weights each term of the first-order expansion of I^* (Eq. (6)) by the topic’s declared publication rate. It is therefore a first-order truncation of the queue-flow simulator I_{dyn} , which reads the same rates, and it is reported as a reference for I_{dyn} rather than as a predictor. It was evaluated after the registered analyses and is exploratory. It is the natural reference for learned approximations of that simulator, and in practice a training-free ranking for queue-flow impact (Section 7.4). Raw-multigraph topic counts (Pubs-raw) and transitive closure without Rule 5 (Reach-R1) are archived in the replication repository. References carry no contrast against **Topo-QoS**, and a learner’s distance from them is reported as such, not as a test. The one exception is the paired comparison between the rate-weighted reference and the learned queue-flow approximation (Section 6.1), which is labeled exploratory.

5.3. Metrics, Protocols and Statistics

Population. Every ranker is scored on the Application set V_{app} .

Metrics. We use Spearman ρ to measure ranking against the simulation oracles. To distinguish between active components and those with zero impact, we report both the full-population Spearman ρ and the active-stratum Spearman $\rho_{>0}$, which only includes components with true impact greater than zero. To identify critical sets, we use **Overlap@K**, the fraction of the true top- K recovered by the predicted top- K , where K is about 20% of $|V_{\text{app}}|$. **PR-AUC**, $F_1@7$, and **nDCG@10** are reported in Supplementary Section S17.

Protocols. In the LOSO setup, we train learned models on eleven scenarios and test on the twelfth, repeating this for all 12 folds and five random seeds. The reported value is the average ρ across seeds. For zero-shot transfer, we evaluate models built on all twelve scenarios across the five system models without any fine-tuning. Training-free rankers are not fitted to any data, so for them, both protocols become per-scenario

analyses on the same twelve scenarios and five models. We include them in the same tables to ensure every row is scored on the same scenarios and labels.

Statistics. The analysis emphasizes effect sizes with confidence intervals rather than a single winner; “matches” means not significantly different, not statistically equivalent (Section 7.5). The summary ρ is the mean Spearman correlation over folds or systems, with a 95% bootstrap interval over folds ($B = 2,000$) [89]; contrasts use paired two-sided Wilcoxon signed-rank tests across folds [90, 91] (smallest attainable $p = 0.00049$), and, because LOSO folds share ten of eleven training scenarios, every registered contrast is also tested with the Nadeau–Bengio corrected resampled t -test [92]. As the effect size we report the number of folds on which a ranker beats its comparator (“won”), the paired probability of superiority; the unpaired Vargha–Delaney $\hat{A}_{12}$ does not apply to fold-paired data. Fisher- z averaging does not change the row order of Table 5; weighting folds by $|V_{\text{app}}|$ changes it for **Topo-QoS** (Section 6.1). Spearman ρ gives ties their average rank; **Overlap@K** breaks ties at the boundary by node order, so Section 6.1 also reports tie-aware recall.

Analysis plan and the status of each result. The protocol is pre-specified in the replication repository with commit timestamps, not lodged with a third-party registry; seventeen numbered amendments were registered, none withheld. Every result has one of three tiers. *Confirmatory*: the plan’s two co-primary contrasts (**HGT-QoS** and **HGT vs. Topo-QoS**, Holm across the two), both null. *Registered secondary*: arms committed before they ran but after the primary null was known, including the matched 2×2 , the hybrids, the reference counts, the derivation value, the full-population queue-flow labels, the sensitivity, direction and corrected-prior arms, and the oracle-aligned-feature, node-order and learning-on-Eq. (7) controls, each Holm-corrected within its family. *Exploratory*: post hoc analyses, the dependency-graph learners’ contrasts (declared exploratory when the arms were registered), the reference reclassification (which changed no figure), and the rate-weighted reference (Eq. (7)) with the attribution of the learned queue-flow approximation. **Plan deviation**: the registered nested selection was replaced by one fixed configuration, with the registered rule run later as a sensitivity check (Section 6.2). Because sequential amendments accumulate family-wise error, an omnibus Holm correction is also applied across the 13 decision-bearing contrasts: the two co-primary contrasts, the two hybrids against **Topo-QoS** and against their base learners, the three effects of the matched 2×2 , and four typing controls (**HGT-QoS** against **GAT-QoS**, **GAT-w** and **HGT-QoS-U**, **HGT** against **GAT**). The hybrid contrasts survive it (Hybrid-GAT $p_{\text{omni}} = 0.019$, Hybrid-HGT $p_{\text{omni}} = 0.041$) and the primary contrast stays null ($p_{\text{omni}} \geq 0.46$); the amendment log (Supplementary Section S27) lists every family. The headline findings of this paper are therefore registered secondary or exploratory, not confirmatory.

Taken together, the rankers allow comparison of representation-centric and model-centric explanations of ranking performance. If performance arises primarily from the dependency representation, analytical and learned rankers built on that representation should perform similarly. If learning adds information, learned rankers should outperform analytical alternatives built on the same dependencies. Section 6 reports the results against these two expectations.

6. Results

All results are reported on the Application population (V_{app}) against the primary oracle $I^*(v)$, and, for the rankers of Table 6, also against the queue-flow oracle $I_{\text{dyn}}(v)$ and the multi-criteria oracle $I_{\text{comp}}(v)$ (Section 4.3). Only the two co-primary contrasts are confirmatory; every other contrast is registered secondary or exploratory (Section 5.3). Per-fold results, secondary strata, and extended protocol notes are in the public experiment pages (Section 5.1). Figure 4 summarizes the main findings.

6.1. RQ1: Ranking Accuracy

Summary. For the reachability and queue-flow oracles, explicit dependency graphs, rather than model complexity, carry most of the ranking signal. The strongest learned model, the graph attention network on the dependency graph (**GAT-P-QoS**, $\rho = 0.748$), approaches without exceeding the direct-dependent reference (0.764; not distinguishable, equivalence not established). On the expensive queue-flow oracle, a gradient-boosted approximation trained on its labels reaches 0.799 because it reads declared publication rates and payload sizes; the rate-weighted reference, which reads the same rates without training, reaches 0.830 and exceeds the learned



Figure 4: Main results for the Application population. (A) Mean Spearman ρ with 95% bootstrap intervals under LOSO (filled circles; Table 5) and zero-shot on five system models (open diamonds); grey hollow markers are references that restate I^* 's rule (InDeg, Reach). GAT-P-QoS is not distinguishable from InDeg, and equivalence within ± 0.05 is not established (TOST $p = 0.17$). (B) Per-fold improvement of GAT-P-QoS and Hybrid-GAT over Topo-QoS. (C) Cell means of the capacity-matched 2×2 on the raw multigraph (Supplementary Table S25): the "QoS" inputs, which include a QoS-weighted in-degree column, raise both models by about 0.07 (not significant after Holm), falling to +0.030 with w_{in} held in both arms (Table 7).

approximation (+0.031, 10 of 12 folds; exploratory), and learners that start from the reference do not improve on it. On the multi-criteria oracle, training-free scores aligned with its terms rank highest and pure neural models drop sharply. The registered primary comparison on the raw multigraph (HGT-QoS vs. Topo-QoS) was null ($\Delta\rho = +0.069$, $p = 0.266$).

Learned rankers are trained on eleven scenarios and scored on the twelfth; training-free rankers are scored on all twelve (26 to 300 Applications, K between 5 and 60). Table 5 reports all of them against I^* , with the reference rankings of Section 5.2 in a separate block: they restate I^* 's rule, so their rows measure the circularity and carry no contrast.

Two patterns emerge. First, reading the dependency graph instead of the raw multigraph improves attention-based learners (GAT-P-QoS 0.748 vs. GAT-QoS 0.635), although the typed transformer becomes unstable on it (HGT-P-QoS 0.514). Second, analytical rankings derived from the same dependency structure are not exceeded by any learned model (0.764–0.808, against at most 0.748). The analyses below examine

both observations. The ordering depends on how folds are weighted: weighting by $|V_{\text{app}}|$ keeps the top five rows but lifts Topo-QoS to 0.596, level with the raw-multigraph learned rankers (0.595–0.606).

Table 5: Main results under LOSO across twelve synthetic architectures (Application population; learned rankers: five seeds, CPU sweeps). $\Delta\rho$ is paired by fold against Topo-QoS, with a bootstrap 95% CI and a two-sided Wilcoxon test; p_{Holm} is within the registered hybrid family; † exploratory. $\rho_{>0}$: Spearman over active components ($I^* > 0$). References restate I^* ’s rule and carry no contrast. Topo-QoS has its articulation term zeroed by a defect (Section 5.2); the indented row restores it, and hybrids retrained on that prior are in Table 9. Further baselines and per-fold values: Supplementary Section S42. Underlined: best predictor per column.

Ranker	LOSO ρ [95% CI]	Active $\rho_{>0}$	$\Delta\rho$ vs Topo-QoS [95% CI]	Won	p (p_{Holm})	Overlap@K
<i>Reference: restatements of I^*’s propagation rule (not predictors)</i>						
Analytic I^*	0.808 [0.755, 0.853]	0.631	—	—	—	0.536
InDeg (direct dependents)	0.764 [0.674, 0.840]	0.516	—	—	—	0.504
Reach (transitive dependents)	0.732 [0.674, 0.782]	0.286	—	—	—	0.344
<i>Training-free baseline (registered comparator)</i>						
Topo-QoS	0.553 [0.443, 0.657]	0.280	—	—	—	0.388
corrected (articulation term restored)	0.533 [0.404, 0.650]	—	—	—	—	—
<i>Learned, on the dependency graph</i>						
GAT-P-QoS	0.748 [0.704, 0.789]	0.440	+0.195 [+0.100, +0.294]	10/12	0.0068 †	0.454
HGT-P-QoS	0.514 [0.392, 0.636]	0.237	−0.039 [−0.155, +0.077]	4/12	0.622 †	0.380
<i>Learned and hybrid, on the raw multigraph</i>						
HGT-QoS	0.622 [0.547, 0.690]	0.312	+0.069 [−0.046, +0.174]	8/12	0.266	0.426
GAT-QoS	0.635 [0.567, 0.696]	0.338	+0.082 [−0.046, +0.201]	7/12	0.233	0.438
Hybrid-HGT	0.657 [0.572, 0.733]	0.345	+0.103 [+0.055, +0.152]	11/12	0.0034 (0.0068)	0.435
Hybrid-GAT	0.683 [0.603, 0.753]	0.362	+0.130 [+0.075, +0.190]	11/12	0.0015 (0.0029)	0.450

The reference level. InDeg is publish–subscribe afferent coupling [61, 62, 63] and counts exactly the Applications reached by I^* ’s first wave; Reach counts the transitive dependents a reachability cascade can visit. Their values (0.764, $\rho_{>0} = 0.516$; 0.732) measure how much of the oracle restating its rule recovers, not predictive skill. The derivation matters for one of them: without Rule 5, Reach drops by 0.058 (9/12 folds, Holm $p = 0.0068$). Untyped raw-graph metrics do not recover the reference (total degree 0.199, reverse PageRank 0.089; PageRank is constant over Applications, which have no incoming raw edges).

The first-order expansion of the oracle. The strongest reference, the closed-form first-order expansion of I^* (Eq. (6)), reaches 0.808 [0.755, 0.853] ($\rho_{>0} = 0.631$): direct subscriber loss is most of what I^* measures, and InDeg is a coarser version of it (it slightly exceeds the expansion on four folds).

Learners on the dependency graph approach the reference. GAT-P-QoS reaches $\rho = 0.748$, 0.016 below the direct-dependent count per seed and 0.007 above it as a five-seed ensemble (0.772, Table 6). Neither difference is significant, and neither shows equivalence: the 90% interval of the per-fold difference is ± 0.076 , so a two one-sided test at the registered margin of ± 0.05 does not pass ($p = 0.17$ per seed, 0.13 for the ensemble). The closeness depends on the degree features (Section 6.2). HGT-P-QoS reaches only 0.514 and is highly seed-dependent (Section 7.5).

Hybrid rankers versus the training-free baseline and their base learners. A hybrid takes $p(v)$ as an input and corrects its logit, so it nests its comparator. Hybrid-HGT (0.657, +0.103, Holm $p = 0.0068$) and Hybrid-GAT (0.683, +0.130, Holm $p = 0.0029$) beat Topo-QoS on 11 of 12 folds, also under the omnibus Holm correction ($p_{\text{omni}} = 0.041$ and 0.019), the Nadeau–Bengio corrected t ($p = 0.019$ and 0.014) and $|V_{\text{app}}|$ weighting (sign-flip $p = 0.038$ and 0.005). Neither differs from its own base learner (Hybrid-HGT vs. HGT-QoS +0.035, $p = 0.73$; Hybrid-GAT vs. GAT-QoS +0.048, $p = 0.30$). Retrained with the articulation defect corrected (Table 9), Hybrid-GAT-AP reaches 0.669 (+0.136, Holm $p = 0.0059$) and Hybrid-HGT-AP 0.640 (+0.107, Holm $p = 0.0073$), each on 11 of 12 folds, and again neither differs from its base learner (+0.034 and +0.018, Holm $p = 0.94$). The hybrid gain therefore belongs to the weak comparator, not to the learned correction. Given the direct-dependent count as their prior, GAT-QoS+InDeg (0.763) and HGT-QoS+InDeg (0.759) gain +0.128 and +0.137 over their base learners (Holm $p = 0.0049$) and land within ± 0.012 of InDeg itself (equivalent at ± 0.05 , TOST $p < 0.001$): a learner handed the reference reproduces it. The prior also hurts transfer (Section 6.3).

Beyond the reachability oracle. Table 6 compares the rankers against two other oracles. On the full-population I_{dyn} , which agrees with I^* at $\rho = 0.711$ [0.625, 0.794], the references outperform all GNNs (first-order expansion 0.706, InDeg 0.664, GNNs 0.496–0.615). Because I_{dyn} shares its topology and first-order

Table 6: Rankers against three oracles, twelve LOSO folds, all 1,321 Applications (registered secondary and exploratory). Partial ρ : Spearman with I_{dyn} after the rank of I^* , or of the first-order expansion $\hat{I}_1^*$, is regressed out of both. Statistic by block: learned GNN rows score the mean of five seeds’ *predictions*, which raises ρ on I^* above Table 5’s per-seed means; rows trained on I_{dyn} (*, approximations of it, not checks of it) report the mean of per-seed ρ ; reference and training-free rows are deterministic. For Topo-QoS, the corrected I^* value is 0.533. Underlined: highest predictor per column (for I^* the GAT-P-QoS ensemble; per seed, GBM-P-QoS $\rightarrow$ dyn* reaches 0.764).

Ranker	I^* ρ	I_{dyn} ρ [95% CI]	Partial $\rho(\cdot, I_{\text{dyn}} I^*)$ [95% CI]	Partial $\rho(\cdot, I_{\text{dyn}} \hat{I}_1^*)$	I_{comp} ρ
<i>Reference: restatements of I^*’s propagation rule (not predictors)</i>					
Analytic I^*	0.808	0.706 [0.629, 0.783]	0.318 [0.234, 0.412]	—	0.636
InDeg	0.764	0.664 [0.562, 0.758]	0.272 [0.188, 0.366]	0.079	0.650
Reach	0.732	0.583 [0.510, 0.651]	0.117 [0.055, 0.184]	0.102	0.302
<i>Reference: restatement of I_{dyn}’s first-order rule (not a predictor; exploratory)</i>					
Rate-weighted (Eq. (7))	0.756	0.830 [0.778, 0.872]	0.578 [0.471, 0.679]	0.573	0.551
<i>Training-free baseline</i>					
Topo-QoS	0.553	0.471 [0.381, 0.559]	0.134 [0.061, 0.204]	−0.091	<u>0.702</u>
<i>Learned GNNs (seed ensemble)</i>					
GAT-P-QoS	<u>0.772</u>	0.615 [0.549, 0.673]	0.111 [0.016, 0.213]	0.182	0.274
Hybrid-GAT	0.702	0.599 [0.514, 0.669]	<u>0.173</u> [0.082, 0.267]	0.056	0.585
Hybrid-HGT	0.672	0.573 [0.497, 0.638]	<u>0.162</u> [0.074, 0.252]	0.029	0.582
HGT-QoS	0.667	0.549 [0.464, 0.622]	0.130 [0.035, 0.224]	0.212	0.201
GAT-QoS	0.647	0.523 [0.436, 0.606]	0.107 [0.018, 0.199]	<u>0.217</u>	0.144
HGT-P-QoS	0.618	0.496 [0.395, 0.601]	0.112 [0.033, 0.190]	0.087	0.334
<i>Learned approximations of I_{dyn} (trained on its labels)</i>					
GBM-P-QoS$\rightarrow$dyn*	0.764	<u>0.799</u>	—	—	0.477
GAT-P-QoS$\rightarrow$dyn*	0.734	0.598	—	—	0.368

effect with I^* , partial correlations show what each ranking keeps beyond them. After removing the rank of I^* , **InDeg** keeps 0.272 [0.188, 0.366], **Reach** 0.117, **Topo-QoS** 0.134 and the GNNs 0.107–0.173, all intervals excluding zero; after removing the first-order expansion instead, **InDeg** keeps 0.079 and the raw-multigraph GNNs 0.212–0.217, so the learners capture some queue-flow signal beyond the unweighted first-order term, though less than the rate-weighted reference (0.573).

The queue-flow oracle separates what the dependency structure provides from what declared rates add. Weighting each term of the first-order expansion (Eq. (6), 0.706 on I_{dyn}) by the topic’s declared publication rate (Eq. (7)) raises it to 0.830 [0.778, 0.872] without training (exploratory). A gradient-boosted model trained on I_{dyn} labels, with counts, the first-order expansion and a feature block that carries declared rates and payload sizes, reaches 0.799: above the unweighted expansion (+0.094, Holm $p = 0.019$) but falls below the rate-weighted reference, which exceeds it by +0.031 [+0.013, +0.049] on 10 of 12 folds (nominal Holm $p = 0.009$; exploratory). An exploratory attribution (Supplementary Section S41) shows where its gain comes from: the declared rate and payload columns alone add +0.097 (9/12 folds, Holm $p = 0.021$), and the seven QoS-derived columns ($w(t)$ -weighted scores and policy shares) add −0.005 ($p = 0.68$). The GNN trained on the same labels, which received neither rates nor payloads, reaches 0.598, below the first-order expansion (−0.108, Holm $p = 0.014$) and no better than the same GNN trained on I^* labels restricted to Applications, a label-support control run in the same sweep (−0.008). **Learning on top of the rate-weighted reference.** The direct test of whether learning adds value beyond the aligned approximation is a learner that starts from it (Table 8, F12). Given Eq. (7) as one more input column, the gradient-boosted approximation reaches 0.830, the formula’s value (+0.000, 5 of 12 folds, Holm $p = 0.97$); trained to correct the formula’s residual, it reaches 0.824 (−0.006, Holm $p = 0.68$); and the GAT trained on I_{dyn} with the formula as its prior gains +0.214 over the same GAT without it (12 of 12 folds) but still falls below the formula (0.812, −0.018, 1 of 12 folds, Holm $p = 0.0029$). No learner improved on the rate-weighted reference; on this simulator, the learned approximations recover the formula’s information and add none.

The multi-criteria oracle favors different rankers. On I_{comp} , learned rankers without a prior perform poorly (0.144–0.334; sum-aggregation GNNs 0.41–0.44), the hybrids keep 0.58, and training-free scores aligned with its fragmentation and flow terms rank highest: **Topo-QoS** 0.702 [0.649, 0.753] (above **InDeg**,

0.650, which is ahead on only 3 of 12 folds, $p = 0.110$) and raw total degree 0.719. As with the references on I^* , this reflects overlap with the oracle’s terms, not predictive skill. No ranker is best across all three oracles. **Identifying the critical set.** In a CI gate the question is whether the true top-ranked items are flagged. Figure 5 shows the share of the true top-20% set on I^* that each ranker places in its top $k\%$, with ties resolved by averaging. At $k = 20\%$, GAT-P-QoS recovers 0.48 of the critical set and Topo-QoS recovers 0.38, matching their $\text{Overlap}@K$; about half of what a top-20% gate flags is outside the true top 20%. To recover 80% of the critical set, GAT-P-QoS must flag the top 40% (0.81), and Topo-QoS does not reach 80% within 50%. The references do only slightly better: at $k = 20\%$, InDeg recovers 0.49 (tie-breaking bounds 0.48–0.50), and 80% recall requires the top 45% by InDeg (0.83) or by the first-order expansion (0.89); only the latter reaches 90% within the top 50%. On the exhaustive I_{dyn} labels (panel B), GAT-P-QoS reaches 0.74 at 40% and 0.82 at 50%, compared to 0.80 at 40% for the InDeg reference; the rate-weighted reference, which restates I_{dyn} ’s first-order rule, recovers 0.81 of that set at 20% and 0.96 at 30% (exploratory). Flagging 40–50% of components to catch 80% of the critical ones would cause alert fatigue, so SaG suits a prioritized triage list for architecture review rather than a blocking gate (Section 7.4).

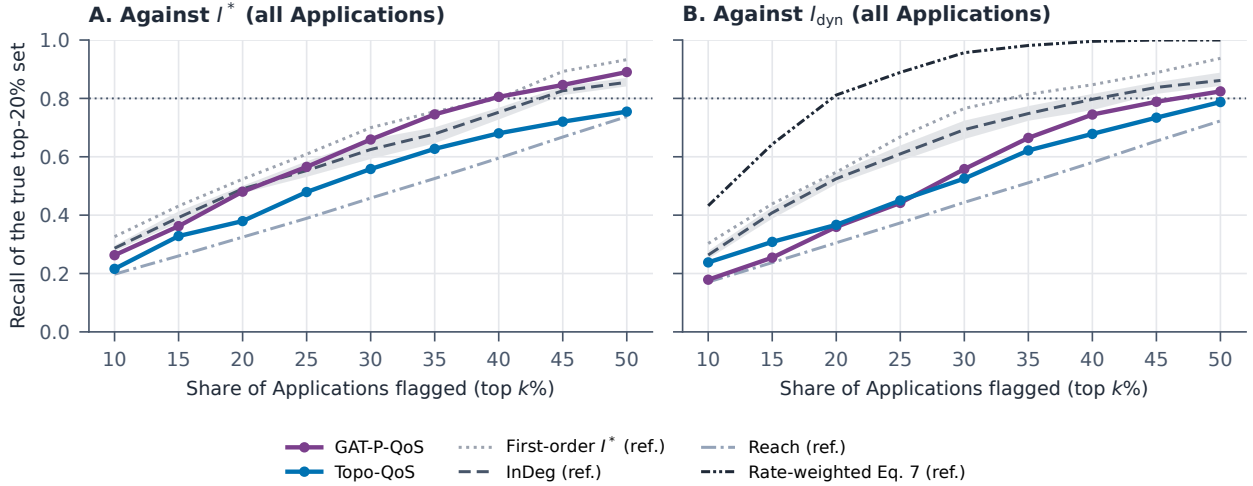


Figure 5: Recall of the true top-20% set (tie-inclusive) by each ranker’s top $k\%$, mean over the twelve LOSO folds, against I^* (A) and I_{dyn} (B), over all 1,321 Applications. Ties are resolved in expectation; grey dashed curves are references restating I^* ’s rule, the black dash-dotted curve in (B) is Eq. (7), and the shaded band gives InDeg’s tie-breaking bounds (Reach’s are much wider, 0.12–0.80 at $k = 20\%$). The dotted line marks 80% recall.

6.2. RQ2: Sources of Predictive Performance

Summary. Attention-based learners improve on the dependency graph, where messages reach the scored Applications from their dependents (0.748 vs. 0.635 on the raw multigraph), and the gain survives two controls: a raw-graph GAT that passes every edge in both directions (+0.072 over it), and the removal of every oracle-aligned feature (+0.231 over the equally stripped reverse-edge control). Its closeness to the direct-dependent count comes from its degree features (−0.136 without them), whereas a sum-aggregation GNN, which can count, needs none: with no node features at all it reaches 0.719. QoS inputs add nothing measurable once weighted in-degree is held, and the typing comparison is uninformative because it ran where messages are inert. Node order before training moves the learned values by about 0.04 (three permutations average 0.729 for GAT-P-QoS), a source of variance rather than a favourable published order.

The “QoS” factor was mostly the weighted in-degree. In the capacity-matched 2×2 on the raw multigraph (Supplementary Table S25; cell means 0.563, 0.548, 0.635, 0.622), relation typing has no effect (−0.014, $p_{\text{Holm}} = 0.940$) and the “QoS” inputs add +0.073 ($p_{\text{Holm}} = 0.127$). The QoS-off arms, however, also zero w_{in} , a QoS-weighted version of InDeg. With w_{in} held in both arms (Table 7, F4), the “QoS” effect drops to +0.030 [−0.023, +0.085] (Holm $p = 0.33$), so more than half of it was the weighted in-degree. Closed-form controls agree: unweighted betweenness (0.591) and constant topic weights (0.595) match or

exceed QoS-weighted betweenness (0.553), and permuting QoS profiles across topics has no effect ($\Delta = -0.006$, $p = 0.733$). Because messages never reach Applications on the raw multigraph, this 2×2 compares per-node models and says nothing about relational typing (Section 7.2).

Table 7: Degree and aggregator arms: twelve LOSO folds, five seeds, one CPU invocation with every comparator re-run to an exact match, Application population. $\Delta\rho$ paired by fold with a bootstrap 95% CI; Holm within each registered family (F1: degree features; F2: aggregator; F4: the 2×2 with w_{in} held). $-\text{deg}$: `in_degree` and w_{in} zeroed; $-\text{deg}^*$: also PageRank, closeness and eigenvector centrality. Zero-shot: mean ρ on the five system models. Registered secondary.

Arm	Comparator	LOSO ρ (I^*)	$\Delta\rho$ [95% CI]	Won	p_{Holm}	$I_{\text{dyn}} / I_{\text{comp}}$	Zero-shot
<i>F1: removing the degree features</i>							
GAT-P-QoS- $-\text{deg}$	GAT-P-QoS (0.748)	0.612	-0.136 [$-0.205, -0.072$]	1/12	0.0049	0.485 / 0.156	0.821
GAT-P-QoS- $-\text{deg}^*$	GAT-P-QoS	0.587	-0.161 [$-0.252, -0.080$]	1/12	0.0049	0.462 / 0.138	0.654
GAT-QoS- $-\text{deg}$	GAT-QoS (0.635)	0.365	-0.270 [$-0.344, -0.199$]	0/12	0.0015	0.295 / -0.047	0.810
<i>F2: sum aggregation (Graph Isomorphism Network with Edge features — GINE, 434,123 parameters) instead of softmax attention</i>							
GIN-P-QoS	GAT-P-QoS	0.732	-0.016 [$-0.055, +0.017$]	7/12	0.910	0.606 / 0.407	0.806
GIN-P-QoS- $-\text{deg}$	GAT-P-QoS- $-\text{deg}$	0.721	$+0.108$ [$+0.027, +0.197$]	8/12	0.157	0.602 / 0.428	0.810
GIN-P-QoS- $-\text{deg}^*$	GAT-P-QoS- $-\text{deg}^*$	0.711	$+0.124$ [$+0.031, +0.230$]	9/12	0.157	0.589 / 0.438	0.776
<i>F4: matched 2×2 with w_{in} held (cells GAT+w_{in} 0.628, HGT+w_{in} 0.569, GAT-QoS 0.635, HGT-QoS 0.622)</i>							
Typing (main effect)	averaged over Q	—	-0.036 [$-0.079, +0.004$]	2/12	0.330	—	—
“QoS” inputs (main effect)	averaged over T	—	$+0.030$ [$-0.023, +0.085$]	9/12	0.330	—	—
Interaction	difference of differences	—	$+0.046$ [$-0.009, +0.101$]	9/12	0.330	—	—

What the dependency-graph learner needs. Each learner receives in-degree and w_{in} as node properties and, on the dependency graph, also aggregates its dependents, so its closeness to `InDeg` could come from either. Removing the two degree columns lowers `GAT-P-QoS` by 0.136, and also removing the three centralities most correlated with in-degree by 0.161 (Table 7, F1). Theory predicts this: softmax attention averages over neighbors and cannot count them, whereas sum aggregation can [77, 78], and a GINE network of the same size keeps 0.721 and 0.711 without the same columns. These aggregator contrasts are not significant after Holm correction ($+0.108$ and $+0.124$, $p_{\text{Holm}} = 0.157$); with every oracle-aligned feature removed, the corresponding contrast is (below). In zero-shot, the degree columns matter little (0.810–0.821 without them), because the system models are ranked mostly by separating inert from active components.

Direction versus dependency semantics. On the raw multigraph a forward GAT never delivers a message to an Application (Section 4.1), so the $+0.113$ gain of `GAT-P-QoS` over `GAT-QoS` could come from edge direction rather than from dependency semantics. A control passes every raw-graph edge in both directions, with shared weights and an unchanged parameter count (`GAT-QoS-R`; Table 9). It reaches 0.676: direction alone recovers $+0.041$ of the gain, which is not significant (Holm $p = 0.064$), and `GAT-P-QoS` still exceeds it by $+0.072$ [$+0.034, +0.113$] on 10 of 12 folds (Holm $p = 0.014$). The derived dependency graph therefore adds beyond edge direction. Reverse edges also cost transfer: `GAT-QoS-R` reaches 0.744 zero-shot, against 0.805 for `GAT-QoS` and 0.806 for `GAT-P-QoS` (Table 10).

Without the oracle-aligned features. Every published learner reads features that compute part of what I^* computes (Section 3.5). Zeroing all of them (in-degree, w_{in} , reverse PageRank, the articulation score, multi-path coupling, fan-out criticality and CDI; Table 8, F11) tests whether the representation gain is carried by those features. It is not. The raw-graph learners collapse without them (`GAT-QoS` to 0.369, the reverse-edge control to 0.378), whereas the dependency-graph GAT keeps 0.610 and beats the reverse-edge control by $+0.231$ (10 of 12 folds, Holm $p = 0.0068$). The oracle-aligned set costs `GAT-P-QoS` 0.138, about what the two degree columns alone cost (0.136), so the remaining features add little. With them removed, sum aggregation beats attention on the dependency graph ($+0.115$, 9 of 12 folds, Holm $p = 0.027$): once the count is not supplied as a feature, the aggregator that can count matters. Without any node features at all, a sum-aggregation GNN on the dependency graph reaches 0.719, 0.045 below `InDeg` (1 of 12 folds): from structure alone it learns most of the dependent count, which the derived graph makes available and the raw multigraph does not.

Node order. ListMLE breaks label ties in input order, which is the generator’s creation order (Section 4.2). Permuting node order before training (Table 8, F13; `GAT-P-QoS-perm`) scores 0.712 against 0.748 (-0.035 , 3 of 12 folds, $p = 0.012$), which triggered the registered flag. Two further permutations registered after that result separate an unlucky draw from a favourable published order: seeds 17, 18 and 19 give 0.712, 0.740

Table 8: Oracle-aligned-feature, learning-on-Eq. (7) and node-order controls: twelve LOSO folds, five seeds, one CPU invocation with every comparator re-run to an exact match, Application population. $\Delta\rho$ is paired by fold against the comparator named, with a bootstrap 95% CI; Holm within each registered family (F11: oracle-aligned features; F12: learners started from Eq. (7)). **-min**: in-degree, w_{in} , reverse PageRank, the articulation score, multi-path coupling, fan-out criticality and CDI zeroed; **-const**: every node feature zeroed. The F12 rows report ρ on I_{dyn} and are trained on its labels; all other rows report ρ on I^* . F13 is a registered gate with a nominal p . $I_{\text{dyn}} / I_{\text{comp}}$: per-seed means from saved predictions. Zero-shot: mean ρ on the five system models (— where not run). Registered secondary.

Arm	Comparator	ρ	$\Delta\rho$ [95% CI]	Won	p_{Holm}	$I_{\text{dyn}} / I_{\text{comp}}$	Zero-shot
<i>F11: every oracle-aligned feature removed (registered)</i>							
GAT-P-QoS-min	GAT-QoS-R-min (0.378)	0.610	+0.231 [+0.128, +0.340]	10/12	0.0068	0.479 / 0.149	0.839
GAT-P-QoS-min	GAT-P-QoS (0.748)	0.610	-0.138 [-0.208, -0.072]	1/12	0.0044	—	—
GIN-P-QoS-min	GAT-P-QoS-min	0.724	+0.115 [+0.038, +0.199]	9/12	0.027	0.606 / 0.423	0.802
<i>Descriptive (no test)</i>							
GAT-QoS-R-min	GAT-QoS-R (0.676)	0.378	-0.298 [-0.387, -0.222]	0/12	—	0.291 / 0.319	0.708
GAT-QoS-min	GAT-QoS (0.635)	0.369	-0.266 [-0.334, -0.201]	0/12	—	0.294 / -0.049	0.794
GIN-P-QoS-const	InDeg (0.764)	0.719	-0.045 [-0.069, -0.024]	1/12	—	0.606 / 0.447	0.858
<i>F12: learners started from Eq. (7), scored on I_{dyn} (registered)</i>							
GBM-P-QoS→dyn+Eq7	Eq. (7) (0.830)	0.830	+0.000 [-0.013, +0.015]	5/12	0.970	—	—
GBM→dyn-resid	Eq. (7)	0.824	-0.006 [-0.019, +0.008]	5/12	0.679	—	—
GAT-P-QoS→dyn+Eq7	Eq. (7)	0.812	-0.018 [-0.025, -0.012]	1/12	0.0029	—	—
<i>F13: node order permuted before training (registered gate; nominal p)</i>							
GAT-P-QoS-perm	GAT-P-QoS	0.712	-0.035 [-0.055, -0.014]	3/12	(0.012)	0.578 / 0.227	—

and 0.734 (mean 0.729), the published value lies 0.019 above their mean, and the per-fold spread across the three permutations averages 0.044. Node order is therefore a source of variance of about 0.04 for the learned rankers, not evidence of a favourable order. Creation order correlates only weakly with the labels ($\rho = -0.20$ to $+0.29$ per fold), and each permutation also changes which nodes the seeded validation split draws, so the spread mixes tie order and validation draw. Learned-ranker differences smaller than this spread, such as the -0.016 between GIN-P-QoS and GAT-P-QoS, should not be read as differences between models.

Table 9: Direction and prior controls: twelve LOSO folds, five seeds, one CPU invocation, Application population; $\Delta\rho$ paired by fold with a bootstrap 95% CI, Holm within each registered family (F8: direction; F9: corrected prior; F10: InDeg prior). **GAT-QoS-R**: GAT-QoS with every raw-graph edge also passed in reverse (shared weights, 429,992 parameters). **-AP**: prior with the articulation term restored; Topo-QoS-AP is the corrected baseline. Re-run comparators reproduce their published per-seed ρ exactly, except Hybrid-HGT (within 1.2×10^{-4} ; not a comparator here). Zero-shot: mean ρ on the five system models. Registered secondary.

Arm	Comparator	LOSO ρ (I^*)	$\Delta\rho$ [95% CI]	Won	p_{Holm}	$I_{\text{dyn}} / I_{\text{comp}}$	Zero-shot
<i>F8: edge direction versus dependency semantics</i>							
GAT-QoS-R	GAT-QoS (0.635)	0.676	+0.041 [+0.000, +0.087]	10/12	0.064	0.547 / 0.299	0.744
GAT-P-QoS	GAT-QoS-R	0.748	+0.072 [+0.034, +0.113]	10/12	0.014	0.597 / 0.280	—
<i>F9: hybrids with the corrected prior</i>							
Hybrid-GAT-AP	Topo-QoS-AP (0.533)	0.669	+0.136 [+0.076, +0.198]	11/12	0.0059	0.577 / 0.561	0.668
Hybrid-HGT-AP	Topo-QoS-AP	0.640	+0.107 [+0.060, +0.153]	11/12	0.0073	0.552 / 0.553	0.702
Hybrid-GAT-AP	GAT-QoS	0.669	+0.034 [-0.047, +0.118]	7/12	0.940	—	—
Hybrid-HGT-AP	HGT-QoS (0.622)	0.640	+0.018 [-0.067, +0.100]	8/12	0.940	—	—
<i>F10: hybrids with the InDeg prior (within ± 0.012 of InDeg, 0.764)</i>							
GAT-QoS+InDeg	GAT-QoS	0.763	+0.128 [+0.063, +0.204]	11/12	0.0049	0.657 / 0.563	—
HGT-QoS+InDeg	HGT-QoS	0.759	+0.137 [+0.074, +0.211]	10/12	0.0049	0.657 / 0.589	—

Message passing on the raw multigraph, and the selection rule. On the raw multigraph no message reaches an Application: a gradient-boosted model on the same per-node features and no graph (**GBM-Feat**) reaches 0.642, level with GAT-QoS, and removing HGT’s reverse pass, its only route to Applications, has no effect (**HGT-QoS-U**, -0.010 , $p = 0.91$). On the dependency graph, **HGT-P-QoS** (0.514) was unstable across seeds in its single untuned configuration (Section 7.2). The plan’s nested hyperparameter selection, applied later on the stage-1 grid, it changes HGT-QoS from 0.622 to 0.677 ($+0.055$) and GAT-P-QoS from 0.748 to 0.693 (-0.054), neither significant (Holm $p = 0.259$); nested HGT-QoS versus Topo-QoS is $+0.123$ on 9 of 12 folds, still not significant (Holm $p = 0.192$). That harness trains on one scenario fewer and stops early on a held-out scenario, and on the three folds where it chose the published hyperparameters it does not reproduce

the published scores (differences up to 0.10), so it compares two protocols, not only two configurations (Section 4.2).

6.3. RQ3: Zero-Shot Transfer to Models of Open-Source Systems

Summary. On five hand-crafted models of open-source systems, the best learned rankers achieve about 0.81 zero-shot ranking for I^* (GAT-P-QoS 0.806, GAT-QoS 0.805), much higher than the training-free baseline (0.526). However, their agreement on the active stratum is weak ($\rho_{>0} \leq 0.342$). The reference rankings are even higher (transitive dependents 0.938, $\rho_{>0} = 0.871$; direct dependents 0.863), since in these models, most of I^* is just reachability. These models are structurally dissimilar; the first author wrote all five, so these results reflect transfer to these models, not to real-world systems. The baseline prior reduces transfer: both hybrids perform worse than their base learners (0.695 and 0.662 compared to 0.760 and 0.805), and correcting the prior’s defect does not change this (0.702 and 0.668; Table 9).

Table 10: Zero-shot transfer to hand-authored models of five open-source systems (Application population), without fine-tuning; “zero-shot” applies only to the learned rows. All rows use the same labels. $\rho_{>0}$: rank correlation over active components. Reference rows restate I^* ’s rule. Intervals are percentile bootstraps over five systems, descriptive only; no test is applied, and the training-free comparator carries the articulation defect of Section 5.2. Underlined: best predictor per column.

Ranker	Evaluation Substrate	Mean ρ [95% CI]	Active $\rho_{>0}$	PR-AUC
<i>Reference: restatements of I^*’s propagation rule (not predictors)</i>				
Reach	Dependency graph	0.938 [0.879, 0.991]	0.871	0.933
InDeg	Dependency graph	0.863 [0.734, 0.952]	0.321	0.752
<i>Training-free baseline</i>				
Topo-QoS	Dependency graph	0.526 [0.357, 0.699]	−0.088	0.474
<i>Learned and hybrid, raw multigraph</i>				
HGT-QoS	Raw multigraph	0.760 [0.714, 0.819]	0.236	0.713
GAT-QoS	Raw multigraph	0.805 [0.759, 0.868]	0.319	0.790
Hybrid-HGT	Raw multigraph	0.695 [0.643, 0.730]	0.210	0.602
Hybrid-GAT	Raw multigraph	0.662 [0.597, 0.727]	0.185	0.600
<i>Learned, dependency graph</i>				
GAT-P-QoS	Dependency graph	<u>0.806</u> [0.785, 0.829]	<u>0.342</u>	<u>0.838</u>

The **Reach** reference reaches $\rho = 0.938$ and $\rho_{>0} = 0.871$ on the five models (0.836–0.997 on the three publish–subscribe systems, 0.966–0.998 on the two converted RPC systems), far above its synthetic-fold values (0.732, $\rho_{>0} = 0.286$). The difference is structural: half of the system models’ Applications have zero simulated impact (0.51, against 0.31 on the folds) and their fan-in is more concentrated (Gini 0.65 vs. 0.50), so separating the inert half already yields a high full-population ρ , which **Reach** does perfectly. On the active stratum, every predictor is weak (learned 0.185–0.342, training-free baseline negative), as is **InDeg** (0.321 [−0.044, 0.687]), while **Reach** keeps 0.871: on these models, transitive reach is nearly what I^* computes. Per system, **GAT-P-QoS** ranges from 0.774 (Train-Ticket) to 0.841 (EdgeX Foundry), a narrow spread that again reflects the inert/active split. All five models are single-modeler and stylized (Section 5.1), so these results show that learned rankers transfer broad topological partitioning across one modeler’s style, not reliable discrimination among active components in new architectures.

6.4. RQ4: Cost

Summary. Measured like for like, on the same graphs in one session (Table 11), one labeling pass of the reachability oracle over the Applications takes 0.01–0.72 s per corpus architecture, the counting path that restates its first wave 17–176× cheaper (median 45×), and the feature extraction every learned ranker needs 4.5–72.5× more (median 16.9×). On generated graphs of up to 5,000 components, the ordering holds, with features at 5.7–18.7× one pass. The queue-flow oracle is the expensive one: 12.7 CPU-hours to label the

corpus. Estimated from the processor’s base power (28 W TDP), replacing the dynamic simulation with the rate-weighted reference (Eq. (7)), computed in at most about a millisecond per architecture, avoids an estimated 355.6 Wh for the corpus. For reachability, static counting remains the greenest choice. Cost should be read together with ranking quality: for both I^* and I_{dyn} , no learned alternative exceeds the analytical approximations, which avoid training and run in milliseconds.

Table 11: Like-for-like cost, every stage timed on the same graph in one session (CPU, one thread, median of 3 runs; once at 4,995 components, where the three-type sweep was not run). Count: Application–Library projection plus **InDeg**. One I^* pass: seed 42, Applications. Sweep: the published five-seed labeling run over Applications, Brokers and Libraries. Features: the analysis producing the learned rankers’ node features. Gate: system-layer analysis with 18 anti-pattern detectors, for reference. Corpus row: minimum–median–maximum over the twelve folds.

$ V $	Count (ms)	One I^* pass (s)	Sweep (s)	Features (s)	Gate (s)	Pass : count	Features : pass
249	2.5	0.26	1.63	1.46	2.05	105×	5.7×
499	6.6	0.79	6.59	6.49	10.01	119×	8.2×
999	13.1	3.90	32.42	30.57	50.92	297×	7.8×
1,998	35.5	18.45	157.86	160.73	275.80	521×	8.7×
4,995	226.2	94.58	—	1,768.81	3,072.60	418×	18.7×
Corpus	0.4–15.6	0.01–0.72	0.09–4.98	0.07–52.25	0.16–81.48	17–45–176×	4.5–16.9–72.5×

Table 11 times each stage on the same graphs in one session, so the ratios compare like with like; the full detection gate and the five-seed labeling sweep are listed for reference, since no ranker needs them. The counting path is the cheapest, one I^* pass over the scored population is next, and the app-layer feature extraction that the learned rankers read is the most expensive. Its articulation and Connectivity Degradation Index phase accounts for 88–91% of it (Enterprise, and the 2,000-component graph); this cost belongs to the chosen feature set, not to learned ranking as such, and a learner restricted to cheaper features would avoid it at an accuracy this study did not measure. Where the I^* ranking itself is wanted, running the oracle is therefore cheaper than approximating it with a learned ranker at every size measured, and the count is cheaper still because it restates the oracle’s first wave. Inference itself is negligible (tens of milliseconds at 2,000 components), the count scales to 10,000 components (1.1 s) and **Reach**, a transitive closure, less well (23 s); timing artifacts are in `docs/research/jss/experiments/rq4-cost.md`. The expensive oracle is I_{dyn} : labeling the corpus took 12.7 CPU-hours (about 7 s per fault run and seed; from 0.01 CPU-hours for Healthcare to 5.88 for Financial Trading), and although fault runs parallelize, a seed-parallel run is bounded by its slowest seed (about 76 min). The rate-weighted reference approximates it in at most about a millisecond per architecture. Hardware power counters were not available to us (RAPL requires elevated privileges on the measurement machine), so energy is estimated as wall-clock time multiplied by the processor’s base power (28 W TDP, Intel Core i7-1370P; `reproduce/energy_estimate.py`). Under this estimate, labeling the corpus with I_{dyn} took about 355.6 Wh, the five-seed I^* labeling sweep 11.1 s (0.086 Wh), and the full detection gate 106.9 s (0.83 Wh). Summed over the twelve folds, the feature extraction that the learned rankers read took 70.0 s (about 0.54 Wh), and the dependency count 0.037 s (about 1 J). A learned approximation could repay its cost only by displacing repeated runs of an expensive simulator such as I_{dyn} , and here the rate-weighted reference already does so without training [21, 22, 23].

7. Discussion, Threats to Validity, and Limitations

7.1. Dependency Representation versus Model Complexity

The central finding of this study is that, for the reachability and queue-flow simulators, explicit dependency representations contributed more to ranking performance than model complexity. The derived dependency graph enabled both analytical and learned approaches. Attention-based learners improved on it relative to the raw multigraph, and the improvement survives a control that passes raw-graph edges in both directions (Section 6.2). It also survives the removal of every feature that computes part of the reachability simulator, without which the raw-graph learners collapse. Yet rankings computed directly from the same graph, the direct-dependent count for the reachability simulator and the rate-weighted first-order expansion for the queue-flow simulator, were not distinguishable from, or exceeded, the best learned models. Pure learners on

the raw multigraph did not outperform the training-free baseline, because messages never reached the scored components. In this corpus, explicit dependency derivation is therefore the principal source of predictive signal, not a preprocessing step for learning. The multi-criteria simulator is the exception: there, raw degree and betweenness, whose terms align with fragmentation, ranked highest.

7.2. When Learning Adds Value, and Relative to What

The study does not show that learning is never useful; it shows where learning adds value and against which comparator. Relative to the training-free baseline, learning helped in two ways, both weak evidence for learning. Hybrids that correct a closed-form prior outperformed that baseline, also with its defect corrected, but not their own base learners, so the gain belongs to the baseline’s weakness rather than to the correction; given the direct-dependent count as their prior, they reproduce it. And learned models transferred zero-shot better than the baseline, mainly by separating inert from active components. Relative to analytical references built on the same dependencies, no learned model showed additional skill: the references were indistinguishable from or exceeded the learned models in-distribution, ranked higher in transfer, and ordering among components that actually propagate failures remained weak for every learned model ($\rho_{>0} \leq 0.342$). The most direct test of learning beyond an aligned analytical ranking, learners that start from the rate-weighted approximation, found no improvement on it (Section 6.1). Learning is therefore likely to contribute most where impact depends on mechanisms that no available analytical approximation captures, a condition this corpus did not exercise. A per-regime breakdown by architecture type is given in Supplementary Section S36.

How message passing contributes. On the raw multigraph, learning used node properties, not relations: gradient-boosted trees on the same per-node features and no graph (**GBM-Feat**) reach $\rho = 0.642$ under LOSO, similar to **GAT-QoS** (0.635); forward GAT convolutions send no message to Applications; and removing HGT’s reverse pass, its only route to them, has no effect. On the derived dependency graph, three layers of message passing gather the dependent neighborhood. Attention still needs the degree columns to approach the dependent count, whereas sum aggregation recovers most of it from structure alone (Section 6.2).

What the study can say about relation typing. Little. On the raw multigraph, where typing was compared at matched capacity, messages never reach Applications, so that comparison is between per-node models and carries no information about relational message passing (Supplementary Table S25). On the dependency graph, where messages do reach Applications, the projection has only two relation types, and the heterogeneous transformer (**HGT-P-QoS**) ran in one untuned configuration (width 100) in which some seeds converged to rankings anti-correlated with the oracle (mean within-fold seed standard deviation 0.254, mean $\rho = 0.514$), while **GAT-P-QoS** was stable (0.030, $\rho = 0.748$). That is an optimization failure of one configuration, not evidence about typing; testing typing would need a substrate with active message passing and more relation types, and a tuned heterogeneous model.

Why the learned queue-flow approximation does not exceed the formula. The gradient-boosted approximation (0.799) beats the unweighted first-order expansion (0.706) because it reads the declared publication rates and payload sizes that I_{dyn} uses. Once rates enter the analytical expansion (Eq. (7)), it reaches 0.830 and exceeds the learned approximation (exploratory), so the deficit is not one of missing inputs. Starting the learners from the formula does not help either: given it as an input, the tree model only matches it, and a GAT given it as a prior stays below it (Section 6.1).

7.3. Restatement versus Prediction

Several of the highest-scoring rankings largely restate a simulator’s propagation rule: the direct-dependent count and the first-order expansion for the reachability simulator, and the rate-weighted first-order approximation for the queue-flow simulator. The reference criterion (Section 4.4) distinguishes recovering a simulator’s definition from predicting its outcomes, and it changes how both analytical and learned results should be read. A learned model that approaches a reference has learned the simulator’s first-order rule, not necessarily anything beyond it. Because simulation-labeled benchmarks are common in architecture analysis, the criterion offers a general check for evaluations of this kind: before crediting a learned model with predictive skill, compare it against the simplest ranking that restates the simulator’s own computation.

7.4. Practical Consequences

Choosing a ranker by simulator cost. If the simulator is cheap, like I^* (0.01–0.72 s per pass over a corpus architecture’s Applications), its ranking can be computed directly from the manifest, and publish–subscribe afferent coupling [61, 62], not distinguishable from any GNN tested, costs a fraction of that; its value is cost, not predictive skill, because it restates the simulator’s first wave. If the simulator is expensive, like I_{dyn} (12.7 CPU-hours for the corpus), the rate-weighted expansion approximates it at $\rho = 0.830$ in milliseconds and exceeds a gradient-boosted approximation trained on its labels (0.799; exploratory). Neither simulator, nor any approximation of it, has been shown to predict real outages.

Choosing a ranker by failure notion. Table 12 summarizes the evidence by oracle. It is conditional on simulation fidelity: where a simulator misses a physical failure mode (partial partitions, disk thrashing), rankings on real clusters could diverge. For reachability impact, running I^* itself is the simplest and cheapest option; no learned ranker is distinguishable from the direct-dependent count. For queue-flow impact, the rate-weighted expansion is the recommended ranker, and declared publication rates and payload sizes, not QoS policies, are the only declared information that carried signal. For fragmentation and throughput loss, betweenness-based scores rank highest because they align with the oracle’s terms. On unfamiliar topologies, learned rankers transfer better than the training-free baseline, but active-stratum agreement is weak and the evidence rests on five models by one author.

Mitigating Alert Fatigue with a Two-Tiered Triage Protocol. Using a top-20% threshold on the direct-dependent count (**InDeg**) recovers about half of the true top-20% set on I^* (0.49), while the rest of the flagged items are not in that set; the best learned ranker does no better (**GAT-P-QoS**, 0.48). Figure 5 shows the trade-off: recovering 80% of the critical set takes the top 45% by **InDeg** (0.83) or the top 40% by **GAT-P-QoS** (0.81), **Topo-QoS** does not reach this level within the top 50%, and no predictor reaches 90% within the top 50%. A gate that catches most critical components would flag nearly half the system, so these rankers are unsuitable as automated pass/fail filters.

We instead propose a two-tiered triage protocol as an inspection heuristic; it is a proposal that needs human-factors and workflow validation before adoption:

1. **Tier 1 (Commit-stage triage):** When reviewing pull requests, compute the direct-dependent count (milliseconds, well under a joule) and the rate-weighted first-order expansion (Eq. (7); about a millisecond; an exploratory result, not confirmed on new data). Show the top 5 components by each as an informational note in the pull request review. This highlights potential cascade risks for peer review without blocking developers or causing alert fatigue.
2. **Tier 2 (Staging and sprint verification):** During nightly builds or sprint architecture reviews, run the simulators on the Tier-1 shortlist within a fixed inspection budget, such as the ten highest-ranked services, and reserve redundant broker routing or chaos fault injection for the components the simulation confirms.

Cost and Green CI/CD. Counting dependents is the cheapest ranker and a single I^* pass the next; the features learned rankers read cost more than the simulator they approximate (Table 11). For the queue-flow simulator, the rate-weighted expansion replaces an estimated 355.6 Wh of simulation for the corpus with milliseconds of computation. Training-free dependency analysis is therefore the lower-energy choice at both commit and staging time [23, 22, 21]; the energy figures are TDP-based estimates, not measurements (Section 7.5).

7.5. Threats to Validity

Construct validity and oracle circularity. All labels are simulated: no ranker is validated against observed failures, and no fault was injected into a running cluster, so the findings establish what can be predicted within the evaluated simulators, not what predicts real failures. The simulators omit runtime dynamics such as buffer bloat, thread-pool exhaustion, garbage-collection pauses and retry storms, and some real outages are sustained by load feedback rather than by structure [47, 48]; there, structural rankings, analytical or learned, may misestimate criticality. Validation on real topologies and real failures is therefore the first priority (Section 7.6). The circularity is analyzed in Section 4.4: the dependency counts restate I^* ,

Table 12: Which ranker the evidence supports, by the failure notion the oracle encodes (registered secondary and exploratory; Section 6; conditional on simulation fidelity). Where a reference restating the simulator’s rule exists (Section 4.4), it is recommended because it is cheaper and at least as accurate; it is not credited with predictive skill.

Failure notion (oracle)	Ranker	Evidence
Reachability cascade (I^*)	Run I^* directly	0.01–0.72 s per pass; cheaper than any learned ranker’s features; best predictor GAT-P-QoS $\rho = 0.748$, not distinguishable from the direct-dependent count; equivalence not established
Queue-flow delivery loss (I_{dyn})	Rate-weighted expansion (training-free; exploratory)	$\rho = 0.830$ in milliseconds (Eq. (7); exploratory); gradient-boosted approximation 0.799; unweighted expansion 0.706; GNN approximation 0.598, which received no rates or payloads
Fragmentation and throughput (I_{comp})	Topo-QoS (training-free)	$\rho = 0.702$, raw total degree 0.719 (its betweenness term aligns with fragmentation; the articulation term is zero); among learned models, only hybrids retain signal ($\rho = 0.585$), while pure learners collapse (≤ 0.334)
Unfamiliar topology	GAT-P-QoS or GAT-QoS	$\rho = 0.806 / 0.805$ vs. baseline 0.526; dependency counts rank higher (0.863–0.938); $\rho_{>0} \leq 0.342$; five single-author models

the rate-weighted expansion restates I_{dyn} , closed-form centrality is term-aligned with I_{comp} , and the learned rankers inherit part of the overlap through the features they read and through being fitted to the oracle. The counts were reclassified as references after all results were final; no number changed. The two other oracles bound, but do not remove, the threat, and all three are well approximated by low-order functions of the dependency graph (a first-order expansion recovers 0.808 of I^* and a rate-weighted one 0.830 of I_{dyn}), so the benchmark cannot exhibit a regime in which learning exceeds an aligned analytical ranking; its absence here is a property of these oracles, not evidence that none exists. The reverse-edge control shares one set of weights across both directions; a direction-typed variant was not tested. Rules 2–4 and 6, which govern brokers and hosts, are untested.

Internal validity, reproducibility, and drift. No ranker reads simulator output (regression tests verify this), although the raw-graph learners read the structural edges the oracles traverse. Capacity, depth and early stopping were held constant across comparison arms; the registered selection rule was not applied to the published learned results (Section 5.3), so every learned result is conditional on one fixed configuration. The learned rankers are seed-sensitive: the mean within-fold standard deviation of ρ is 0.099 for **HGT-QoS**, 0.254 for **HGT-P-QoS** and 0.024–0.030 for the GATs, and one of 60 **HGT-QoS** runs and four of 60 **HGT-P-QoS** runs converge to rankings anti-correlated with the oracle (none for either GAT). They are kept in the per-seed means, as registered, so the **HGT** rows reflect training instability as much as typing. Across revision cycles and compute devices, the absolute value of a learned cell drifted by up to 0.172, so learned cells from different sweeps or devices should not be compared. Every learned-versus-learned contrast this paper tests is instead paired within one CPU sweep in which the re-run comparators reproduce their published per-seed ρ (exactly, or within 1.2×10^{-4} for one); the exception, the nested-selection check, compares two harnesses and is reported as a protocol comparison (Section 4.2). Paired contrasts were not repeated on a second device. Node order is a further source of variance: three permutations of the input order move **GAT-P-QoS** by about 0.04 per fold around a mean 0.019 below the published value (Section 6.2), so learned-ranker differences of that size are not interpretable. CDI uses hash-dependent tie-breaking, so **PYTHONHASHSEED=0** is fixed, and all reported values are reconciled mechanically against released artifacts.

External validity and the single-modeler threat. Synthetic topologies come from a single generator family, which may induce parametric regularities not representative of heterogeneous industrial architectures. The five system models are hand-authored approximations (22–41 Applications) of open-source projects, each written by the first author from public documentation. All five **RQ3** results are exposed to this threat, and it is not mitigated: no second modeler authored any model independently, and no inter-modeler reliability or agreement metric is reported. The system models also differ structurally from the synthetic folds: a

larger share of their Applications has zero simulated impact, and their fan-in is more concentrated, favoring **Reach** and inflating full-population ρ . Two of the five (Train-Ticket and Online Boutique) are RPC systems re-expressed as publish-subscribe meshes; the translation keeps bidirectional dataflow but drops synchronous blocking, timeouts and thread starvation. No topology in the study was extracted from a real deployment manifest. We did not evaluate architectures with tens of thousands of components; we timed the counting path up to 10,000 components (Section 6.4), and we did not evaluate the learned rankers or the oracles. We did not reproduce published learned criticality models (FINDER, DrBC) because they do not support typed multigraphs or pub-sub semantics.

Sustainability and energy estimation validity. Energy figures (Section 6.4) are wall-clock time multiplied by the processor’s 28 W base power (`reproduce/energy_estimate.py`), not readings from RAPL counters or power meters. They ignore frequency scaling, turbo power above base (up to 64 W on this processor), DRAM and idle power, and attribute whole-package power to single-threaded work. They support order-of-magnitude comparisons between milliseconds and hours of computation, not energy or emissions accounting.

Conclusion validity. Only the plan’s two co-primary contrasts are confirmatory, and both were null (Section 5.3); everything else was designed after that result was known. LOSO folds share ten of eleven training scenarios, so Wilcoxon p -values are nominal and anti-conservative; the Nadeau–Bengio corrected test leaves the hybrid contrasts significant and the primary null. Twelve folds from one generator make the effective sample small, bootstrap CIs coarse, and the study underpowered for equivalence at ± 0.05 (the 90% interval of the **GAT-P-QoS-InDeg** difference is ± 0.076); “matches” therefore means not significantly different. Weighting folds by $|V_{\text{app}}|$ raises **Topo-QoS** to the level of the raw-graph learned rankers.

7.6. Limitations and Future Work

Six directions follow from the study’s central question, in order of priority:

1. **Real topologies and real failures:** automatic extraction of SaG models from real artifacts (ROS 2 launch files with HAROS or ROSDiscover [59, 60], docker-compose and Kubernetes manifests), so that the derivation is exercised on real manifests, followed by controlled fault injection on ROS 2, Kafka and MQTT deployments (for example with Chaos Mesh or ChaosBlade), reference systems with known faults such as Train-Ticket [33], and incident data from industrial publish-subscribe systems. This should be coupled with runtime distributed tracing standards (e.g., OpenTelemetry, W3C Trace Context) to empirically validate whether static declared dependency paths faithfully reflect dynamic runtime message flow trajectories under normal and degraded conditions.
2. **A confirmation corpus:** scenarios generated after the analysis is frozen, on which the registered secondary and exploratory claims of this paper, all specified after the primary null, are re-tested.
3. **Infrastructure and broker validation:** extending cascade-impact evaluation beyond Applications to message brokers, topics, and compute hosts (evaluating Rules 2–4 and 6), which were outside the Application-scoped evaluation of this study.
4. **Independent modelers:** replication of the five system models by additional modelers, with inter-modeler agreement reported.
5. **Alternative impact models:** testing whether the conclusions hold for simulators with different failure mechanisms, including synchronous call semantics, which the current dependency rules do not model, and oracles with non-first-order mechanisms such as backpressure onto publishers, retry amplification and heterogeneous service capacity, where an aligned analytical approximation may not exist.
6. **When learning exceeds analytical methods:** identifying impact-generation mechanisms whose predictive signal no aligned analytical approximation captures, and testing learners there; in this corpus, learners started from the rate-weighted approximation did not improve on it.

Further extensions are integration with service mesh telemetry and distributed tracing (e.g., OpenTelemetry, Jaeger) for runtime dependency drift detection; hardware-measured energy accounting (RAPL counters required privileges unavailable for this study); a queue-flow run with QoS policies disabled; a tie-aware listwise loss with scenario-level validation; a full hyperparameter search for every learned model; and validation of the proposed explanation layer.

8. Conclusion

This paper presented Software-as-a-Graph (SaG), a pre-deployment framework that derives explicit dependency graphs from publish-subscribe deployment manifests and supports analytical, hybrid, and learned approaches to cascade-impact ranking. Using twelve synthetic architectures, three failure simulators and five stylized models of open-source systems, the study examined when graph learning improves on rankings derived directly from explicit architectural dependencies. The primary registered contrast was null, and the findings below are registered secondary or exploratory.

For the reachability and queue-flow simulators, explicit dependency representations provided most of the predictive signal. Attention-based learners benefited from the derived dependency graph, and that gain survives a control for edge direction, and the removal of every feature that computes part of the reachability simulator, but simple analytical rankings built on the same dependencies were not distinguishable from, or exceeded, every learned model: counting direct dependents on the reachability simulator, and a rate-weighted first-order expansion, computed in milliseconds, on the queue-flow simulator, whose labels took 12.7 CPU-hours to generate. Learners that start from the rate-weighted approximation matched it at best.

Learning added value only relative to a weak comparator: hybrids beat the training-free baseline, which they nest, but not their own base learners, and learned models transferred zero-shot better than that baseline, while dependency-based references remained stronger and ordering among components that actually propagate failures remained weak. For researchers, the results show why graph-learning approaches should be evaluated against analytical alternatives aligned with the simulator, and why prediction should be distinguished from restatement of a simulator’s rule. For practitioners, training-free dependency analysis is effective and inexpensive support for prioritizing architectural review. Learned models can be expected to add value only where no aligned analytical approximation exists; all three simulators here admit one, so this corpus could not exercise that condition.

These findings concern simulated failures on synthetic architectures and single-modeler system models, and their validation against real outages is the natural next step (Section 7.6). More broadly, the study suggests that the central challenge in architecture-level cascade-impact analysis is not selecting increasingly sophisticated learning algorithms, but deriving meaningful dependency representations and understanding when learning adds value beyond them. By making this comparison explicit and by releasing a reproducible benchmark with reconciled results, Software-as-a-Graph provides a foundation for more principled evaluation of analytical and learned methods for pre-deployment architecture analysis.

Declarations

CRedit Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision.

Declaration of Competing Interest. The authors declare no competing financial interests or personal relationships that could have influenced this work.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data Availability. The complete replication package—including datasets, simulation harnesses, model checkpoints, analysis scripts, and reproduction workflows—is archived on Zenodo under DOI 10.5281/zenodo.23045204 [93]. The software [94], environment specifications, and reproduction documentation are publicly available on GitHub at an immutable commit, <https://github.com/onuralpyigit/software-as-a-graph/tree/f3352f0c7cdf66ff77845b5b69624db539e82ed5/docs/research/jss/experiments>. The script `reproduce/reconcile_manuscript.py` mechanically verifies 1,814 reported figures against the released artifacts.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process. During the preparation of this work, the authors used Anthropic’s Claude Opus 5 and Opus 5.5, and Google’s Gemini Flash 3.8, to suggest revisions to the manuscript text, which the authors evaluated, edited, and verified against the results; for LaTeX typesetting and formatting assistance; and for developing analysis scripts in the replication package. They used Grammarly for grammar checking and language copy-editing. The authors reviewed and edited the output as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors’ own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074. doi:10.1126/scirobotics.abm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data Distribution Service (DDS), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O'Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131. doi:10.1145/857076.857078.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382. doi:10.1038/35019019.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [15] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [16] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [18] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [19] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, R. Padhye, Service-level fault injection testing, in: *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*, ACM, 2021, pp. 388–402. doi:10.1145/3472883.3487005.
- [20] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, X. Li, Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments, in: *Proceedings of The Web Conference 2021 (WWW '21)*, ACM, 2021, pp. 3087–3098. doi:10.1145/3442381.3449905.
- [21] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [22] R. Verdechia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [23] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, *Communications of the ACM* 63 (12) (2020) 54–63. doi:10.1145/3381831.
- [24] S. M. Yacoub, H. H. Ammar, A methodology for architecture-level reliability risk analysis, *IEEE Transactions on Software Engineering* 28 (6) (2002) 529–547. doi:10.1109/TSE.2002.1010058.
- [25] T. Zimmermann, N. Nagappan, Predicting defects using network analysis on dependency graphs, in: *Proceedings of the 30th International Conference on Software Engineering (ICSE '08)*, ACM, 2008, pp. 531–540. doi:10.1145/1368088.1368161.
- [26] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324. doi:10.1038/s42256-020-0177-2.
- [27] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568. doi:10.1145/3357384.3357979.
- [28] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2025, pp. 19–30. doi:10.1145/3676151.3719362.
- [29] R. Premraj, K. Herzig, Network versus code metrics to predict defects: A replication study, in: *2011 International Symposium on Empirical Software Engineering and Measurement (ESEM)*, IEEE, 2011, pp. 215–224. doi:10.1109/ESEM.2011.30.
- [30] V. J. Hellendoorn, P. Devanbu, Are deep neural networks the best choice for modeling source code?, in: *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017, pp. 763–773. doi:10.1145/3106237.3106290.
- [31] F. Errica, M. Podda, D. Bacciu, A. Micheli, A fair comparison of graph neural networks for graph classification, in: *International Conference on Learning Representations (ICLR)*, 2020, <https://openreview.net/forum?id=HygDF6NFPB>.
- [32] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [33] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260. doi:10.1109/TSE.2018.2887384.
- [34] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204. doi:10.1016/S0166-5316(01)00034-7.
- [35] S. Becker, H. Koziol, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22. doi:10.1016/j.jss.2008.03.066.
- [36] W. Abdelmoez, D. M. Nassar, M. Shereshevsky, N. Gradetsky, R. Gunnalan, H. H. Ammar, B. Yu, A. Mili, Error propagation in software architectures, in: *Proc. 10th IEEE Int. Software Metrics Symp. (METRICS)*, 2004, pp. 384–393.

- [37] P. Popic, D. Desovski, W. Abdelmoez, B. Cukic, Error propagation in the reliability analysis of component based systems, in: Proc. 16th IEEE Int. Symp. on Software Reliability Engineering (ISSRE), 2005, pp. 53–62.
- [38] V. Cortellessa, V. Grassi, A modeling approach to analyze the impact of error propagation on reliability of component-based systems, in: Proc. 10th Int. Symp. on Component-Based Software Engineering (CBSE), Vol. 4608 of LNCS, Springer, 2007, pp. 140–156. doi:10.1007/978-3-540-73551-9_10.
- [39] M. Hiller, A. Jhumka, N. Suri, EPIC: Profiling the propagation and effect of data errors in software, IEEE Transactions on Computers 53 (5) (2004) 512–530.
- [40] Y. Papadopoulos, J. A. McDermid, Hierarchically performed hazard origin and propagation studies, in: Computer Safety, Reliability and Security (SAFECOMP), Vol. 1698 of LNCS, Springer, 1999, pp. 139–152. doi:10.1007/3-540-48249-0_13.
- [41] J. Delange, P. Feiler, Architecture fault modeling with the AADL Error-Model Annex, in: Proc. 40th EUROMICRO Conf. on Software Engineering and Advanced Applications (SEAA), 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [42] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
- [43] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020.
- [44] C. Lee, T. Yang, Z. Chen, Y. Su, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023, pp. 1750–1762. doi:10.1109/ICSE48619.2023.00150.
- [45] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, ACM Transactions on Software Engineering and Methodology (2025). doi:10.1145/3715005.
- [46] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, Y. Ding, J. He, C. Xu, Characterizing microservice dependency and performance: Alibaba trace analysis, in: Proceedings of the ACM Symposium on Cloud Computing (SoCC '21), ACM, 2021, pp. 412–426. doi:10.1145/3472883.3487003.
- [47] N. Bronson, A. Aghayev, A. Charapko, T. Zhu, Metastable failures in distributed systems, in: Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '21), ACM, 2021, pp. 221–227. doi:10.1145/3458336.3465286.
- [48] L. Huang, M. Magnusson, A. B. Muralikrishna, S. Estyak, R. Isaacs, A. Aghayev, T. Zhu, A. Charapko, Metastable failures in the wild, in: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22), USENIX Association, 2022, pp. 73–90. URL <https://www.usenix.org/conference/osdi22/presentation/huang-lexiang>
- [49] S. A. Böhner, R. S. Arnold, Software Change Impact Analysis, IEEE Computer Society Press, Los Alamitos, CA, 1996.
- [50] S. Esparrachiar, T. Reilly, A. Rentz, Tracking and controlling microservice dependencies, ACM Queue 16 (4) (2018). doi:10.1145/3277539.3277541.
- [51] X. Yang, K. Tang, X. Yao, A learning-to-rank approach to software defect prediction, IEEE Transactions on Reliability 64 (1) (2015) 234–246.
- [52] R. G. Sargent, Verification and validation of simulation models, Journal of Simulation 7 (1) (2013) 12–24. doi:10.1057/jos.2012.20.
- [53] M. Kitsak, L. K. Gallos, S. Havlin, F. Liljeros, L. Muchnik, H. E. Stanley, H. A. Makse, Identification of influential spreaders in complex networks, Nature Physics 6 (11) (2010) 888–893. doi:10.1038/nphys1746.
- [54] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann, Shortcut learning in deep neural networks, Nature Machine Intelligence 2 (11) (2020) 665–673. doi:10.1038/s42256-020-00257-z.
- [55] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.
- [56] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.
- [57] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.
- [58] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.
- [59] A. Santos, A. Cunha, N. Macedo, Static-time extraction and analysis of the ROS computation graph, in: 2019 Third IEEE International Conference on Robotic Computing (IRC), IEEE, 2019. doi:10.1109/IRC.2019.00018.
- [60] C. S. Timperley, T. Dürschmid, B. Schmerl, D. Garlan, C. Le Goues, ROSDiscover: Statically detecting run-time architecture misconfigurations in robotics systems, in: 2022 IEEE 19th International Conference on Software Architecture (ICSA), 2022, pp. 112–123. doi:10.1109/ICSA53651.2022.00019.
- [61] R. C. Martin, Agile Software Development: Principles, Patterns, and Practices, Prentice Hall, 2003.
- [62] D. Rud, A. Schmietendorf, R. R. Dumke, Product metrics for service-oriented infrastructures, in: Applied Software Measurement: Proceedings of the International Workshop on Software Metrics and DASMA Software Metrik Kongress (IWSM/MetriKon 2006), Shaker Verlag, Aachen, Germany, 2006, pp. 161–174.
- [63] J. Bogner, S. Wagner, A. Zimmermann, Automatically measuring the maintainability of service- and microservice-based systems: A literature review, in: Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement (IWSM Mensura '17), ACM, 2017, pp. 107–115. doi:10.1145/3143434.3143443.
- [64] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
- [65] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.
- [66] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.
- [67] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.
- [68] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
- [69] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.
- [70] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.
- [71] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.
- [72] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.

- [73] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024, arXiv:2402.02827.
- [74] S. K. Maurya, X. Liu, T. Murata, Graph neural networks for fast node ranking approximation, *ACM Transactions on Knowledge Discovery from Data* 15 (5) (2021) 78:1–78:32. doi:10.1145/3446217.
- [75] N. Park, A. Kan, X. L. Dong, T. Zhao, C. Faloutsos, Estimating node importance in knowledge graphs using graph neural networks, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*, ACM, 2019, pp. 596–606. doi:10.1145/3292500.3330855.
- [76] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [77] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks?, in: *International Conference on Learning Representations (ICLR)*, 2019.
- [78] G. Corso, L. Cavalleri, D. Beaini, P. Liò, P. Veličković, Principal neighbourhood aggregation for graph nets, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 13260–13271.
- [79] Z. Chen, L. Chen, S. Villar, J. Bruna, Can graph neural networks count substructures?, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 10383–10395.
- [80] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710. doi:10.1145/3366423.3380027.
- [81] Q. Lv, M. Ding, Q. Liu, Y. Chen, W. Feng, S. He, C. Zhou, J. Jiang, Y. Dong, J. Tang, Are we really making much progress? Revisiting, benchmarking, and refining heterogeneous graph neural networks, in: *Proc. 27th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD)*, 2021, pp. 1150–1160. doi:10.1145/3447548.3467350.
- [82] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, arXiv preprint arXiv:1811.05868 [preprint] (2018). doi:10.48550/arXiv.1811.05868.
- [83] Q. Huang, H. He, A. Singh, S.-N. Lim, A. R. Benson, Combining label propagation and simple models out-performs graph neural networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2021.
- [84] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [85] G. H. Hardy, J. E. Littlewood, G. Pólya, *Inequalities*, 2nd Edition, Cambridge University Press, 1952.
- [86] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [87] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: *Proc. 25th Int. Conf. on Machine Learning (ICML)*, 2008, pp. 1192–1199.
- [88] Team SimPy, Simpy: Discrete event simulation for Python [software], Version 4.1.1, <https://pypi.org/project/simpy/4.1.1/> (accessed 9 September 2026) (2023).
- [89] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [90] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [91] A. Arcuri, L. Briand, A practical guide for using statistical tests to assess randomized algorithms in software engineering, in: *Proc. 33rd Int. Conf. on Software Engineering (ICSE)*, 2011, pp. 1–10. doi:10.1145/1985793.1985795.
- [92] C. Nadeau, Y. Bengio, Inference for the generalization error, *Machine Learning* 52 (2003) 239–281. doi:10.1023/A:1024068626366.
- [93] İ. O. Yiğit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.23045204> (2026). doi:10.5281/zenodo.23045204.
- [94] İ. O. Yiğit, F. Buzluca, Software-as-a-graph [software], GitHub, <https://github.com/onuralpyigit/software-as-a-graph/tree/f3352f0c7cdf66ff77845b5b69624db539e82ed5> (accessed 5 October 2026) (2026).